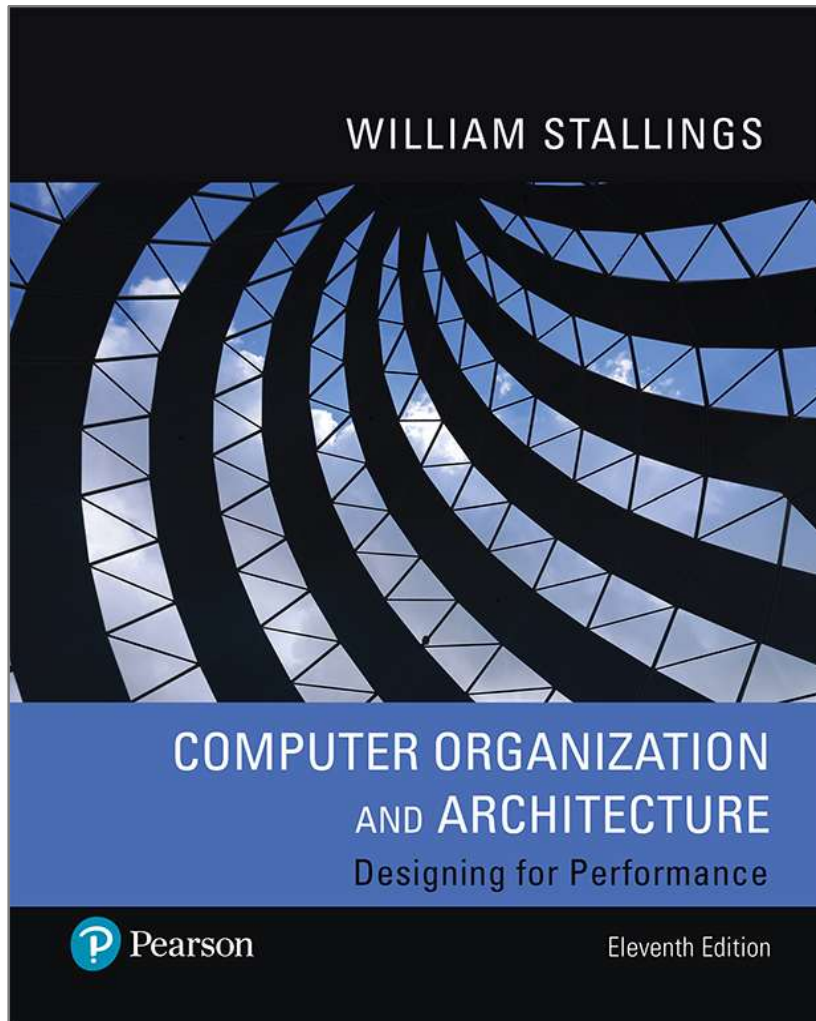


Computer Organization and Architecture

Designing for Performance

11th Edition



Chapter 13

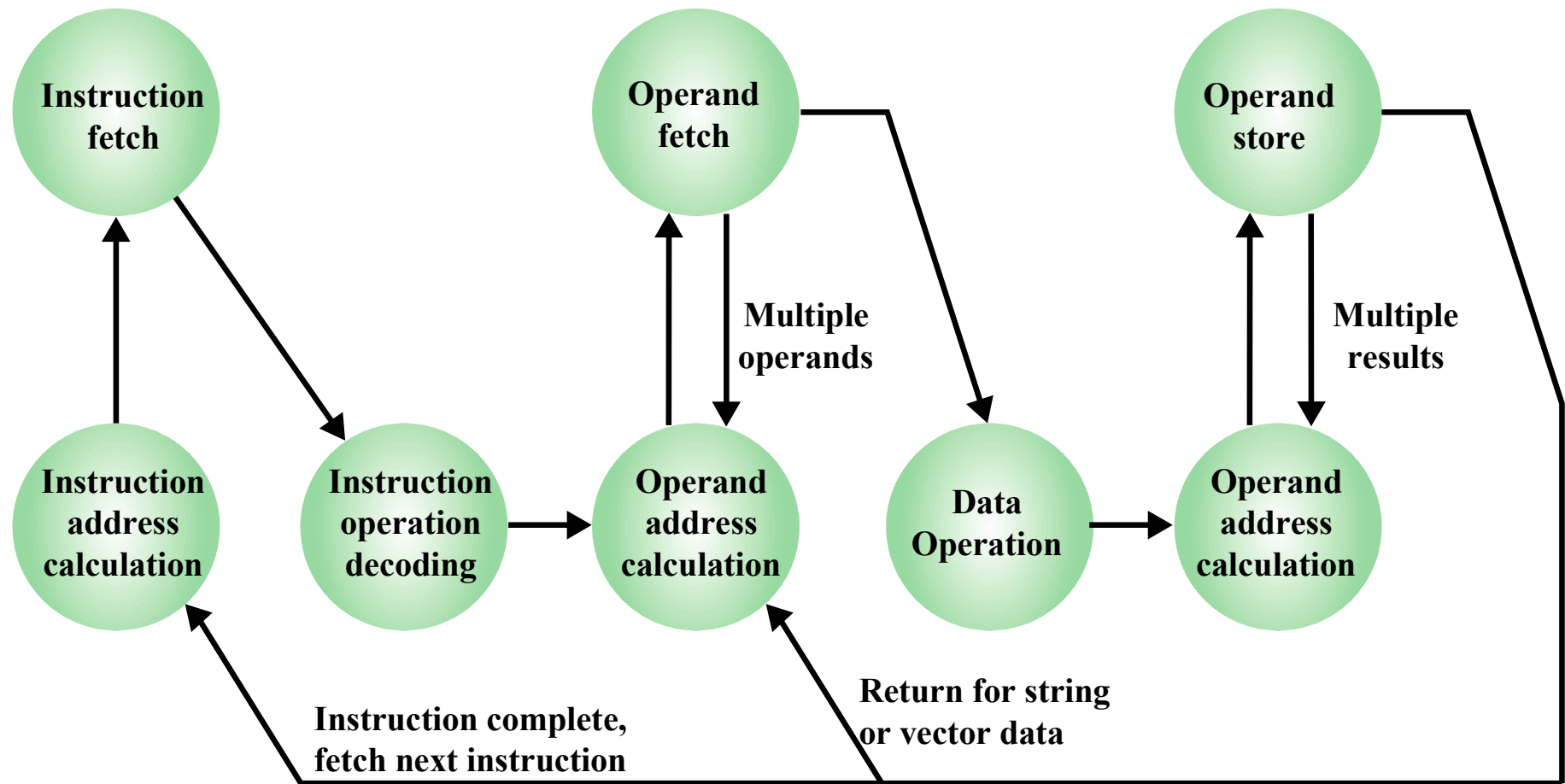
Instruction Sets:
Characteristics and
Functions

Machine Instruction Characteristics

- The operation of the processor is determined by the instructions it executes, referred to as *machine instructions* or *computer instructions*
- The collection of different instructions that the processor can execute is referred to as the processor's *instruction set*
- Each instruction must contain the information required by the processor for execution

Figure 13.1

Instruction Cycle State Diagram



Source and result operands can be in one of four areas:

1) Main or virtual memory

- As with next instruction references, the main or virtual memory address must be supplied

2) I/O device

- The instruction must specify the I/O module and device for the operation. If memory-mapped I/O is used, this is just another main or virtual memory address

3) Processor register

- A processor contains one or more registers that may be referenced by machine instructions.
- If more than one register exists each register is assigned a unique name or number and the instruction must contain the number of the desired register

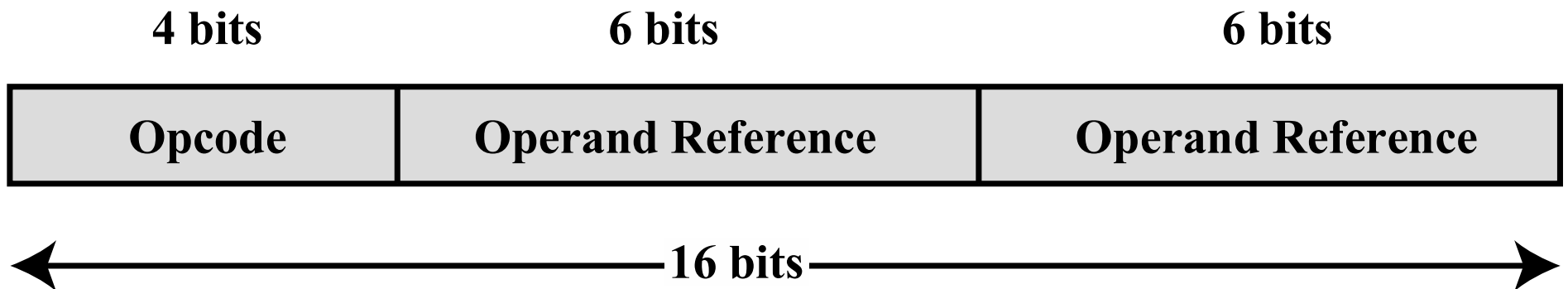
4) Immediate

- The value of the operand is contained in a field in the instruction being executed

Figure 13.2

A Simple Instruction Format

- Within the computer each instruction is represented by a sequence of bits
- The instruction is divided into fields, corresponding to the constituent elements of the instruction



Instruction Representation

- Opcodes are represented by abbreviations called *mnemonics*
- Examples include:
 - ADD Add
 - SUB Subtract
 - MUL Multiply
 - DIV Divide
 - LOAD Load data from memory
 - STOR Store data to memory
- Operands are also represented symbolically
- Each symbolic opcode has a fixed binary representation
 - The programmer specifies the location of each symbolic operand

Instruction Types

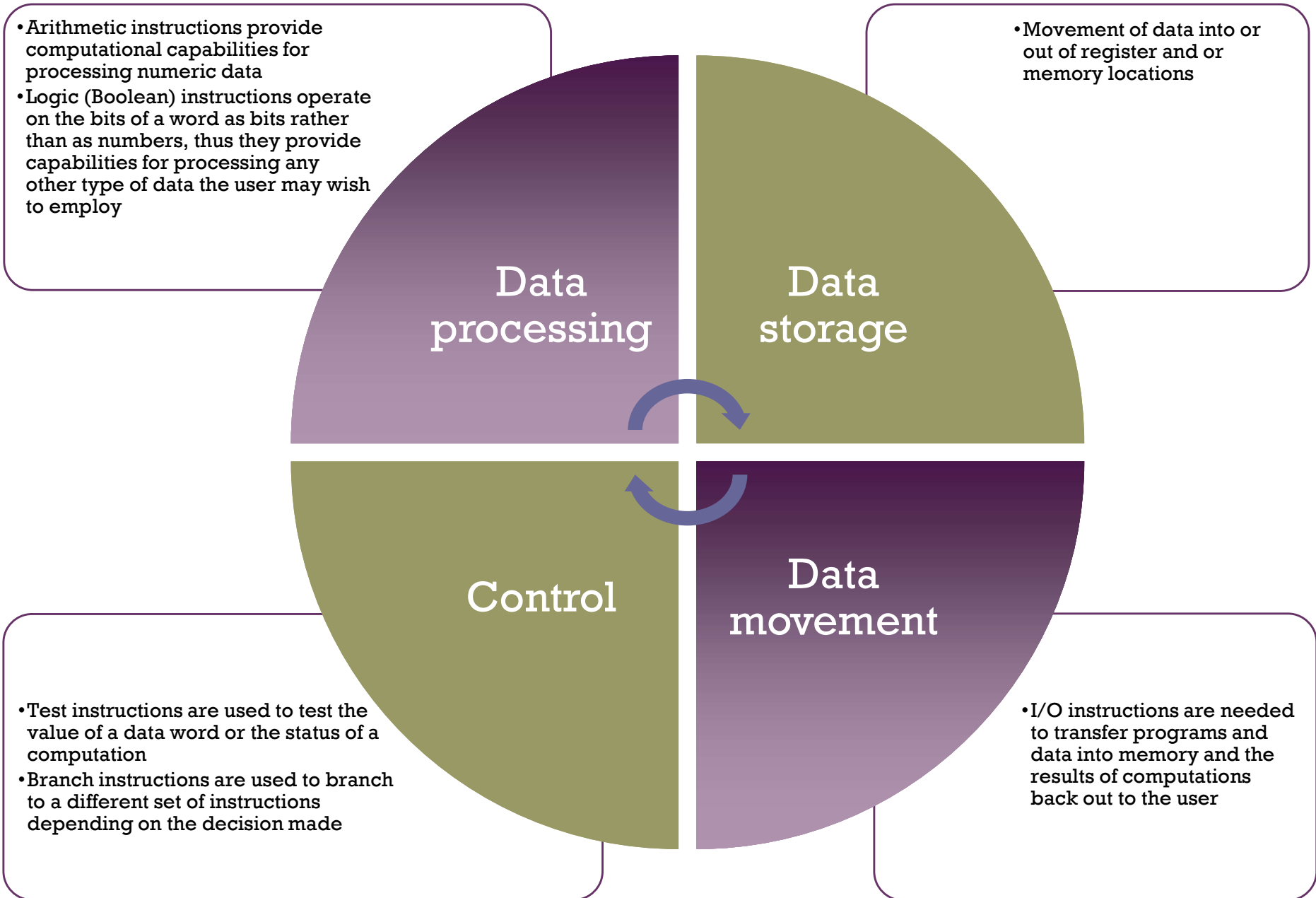


Figure 13.3

Programs to Execute $Y = \frac{A - B}{C + (D \times E)}$

<u>Instruction</u>		<u>Comment</u>
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

(a) Three-address instructions

<u>Instruction</u>		<u>Comment</u>
MOVE	Y, A	$Y \leftarrow A$
SUB	Y, B	$Y \leftarrow Y - B$
MOVE	T, D	$T \leftarrow D$
MPY	T, E	$T \leftarrow T \times E$
ADD	T, C	$T \leftarrow T + C$
DIV	Y, T	$Y \leftarrow Y \div T$

<u>Instruction</u>		<u>Comment</u>
LOAD	D	$AC \leftarrow D$
MPY	E	$AC \leftarrow AC \times E$
ADD	C	$AC \leftarrow AC + C$
STOR	Y	$Y \leftarrow AC$
LOAD	A	$AC \leftarrow A$
SUB	B	$AC \leftarrow AC - B$
DIV	Y	$AC \leftarrow AC \div Y$
STOR	Y	$Y \leftarrow AC$

Table 13.1

Utilization of Instruction Addresses (Nonbranching Instructions)

Number of Addresses	Symbolic Representation	Interpretation
3	OP A, B, C	$A \leftarrow B \text{ OP } C$
2	OP A, B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow (T - 1) \text{ OP } T$

AC = accumulator

T = top of stack

(T - 1) = second element of stack

A, B, C = memory or register locations

Instruction Set Design

Very complex because it affects so many aspects of the computer system

Defines many of the functions performed by the processor

Programmer's means of controlling the processor

Fundamental design issues:

Operation repertoire

- How many and which operations to provide and how complex operations should be

Data types

- The various types of data upon which operations are performed

Instruction format

- Instruction length in bits, number of addresses, size of various fields, etc.

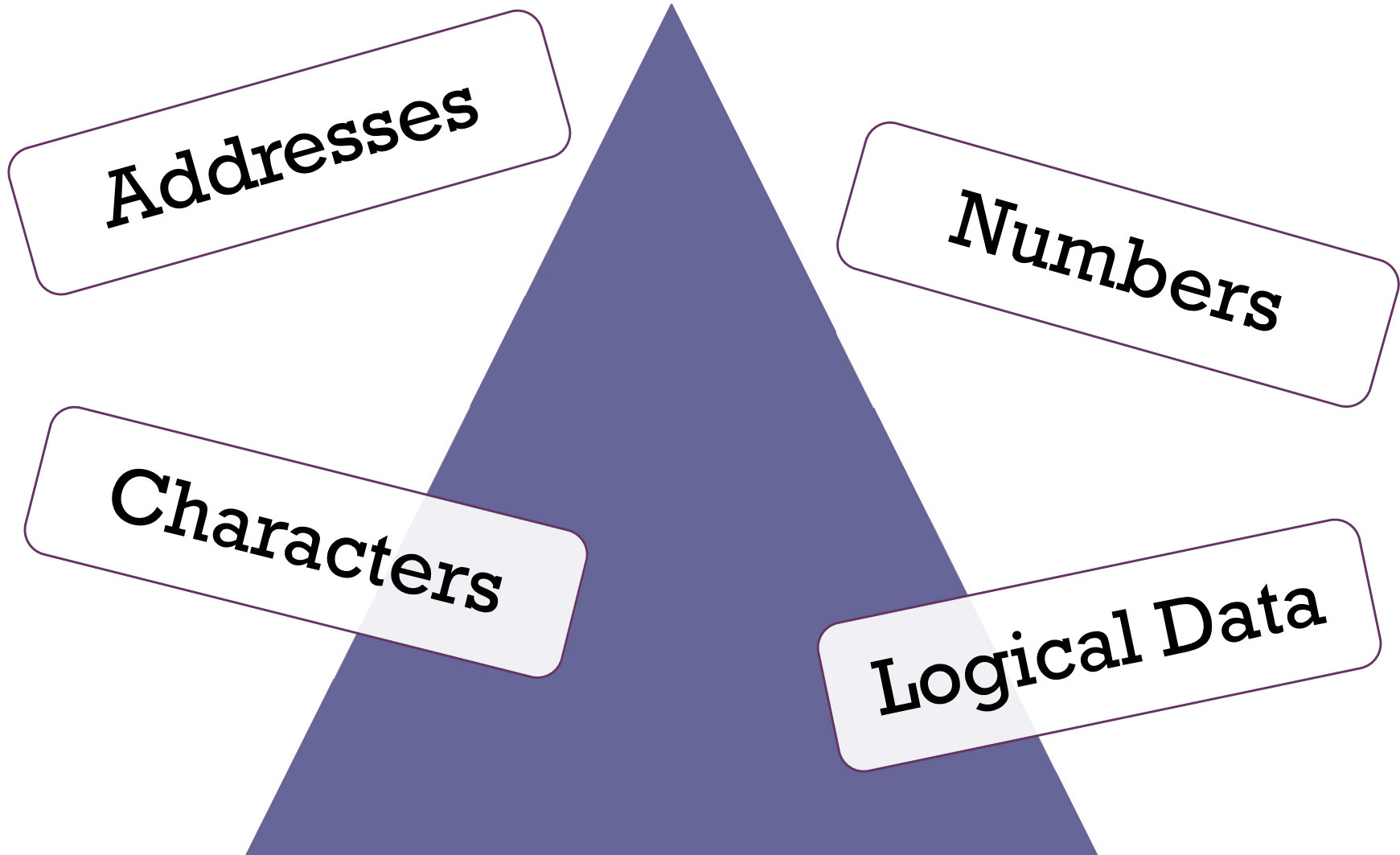
Registers

- Number of processor registers that can be referenced by instructions and their use

Addressing

- The mode or modes by which the address of an operand is specified

Types of Operands



Numbers

- All machine languages include numeric data types
- Numbers stored in a computer are limited:
 - Limit to the magnitude of numbers representable on a machine
 - In the case of floating-point numbers, a limit to their precision
- Three types of numerical data are common in computers:
 - Binary integer or binary fixed point
 - Binary floating point
 - Decimal
- Packed decimal
 - Each decimal digit is represented by a 4-bit code with two digits stored per byte
 - To form numbers 4-bit codes are strung together, usually in multiples of 8 bits

Characters

- A common form of data is text or character strings
- Textual data in character form cannot be easily stored or transmitted by data processing and communications systems because they are designed for binary data
- Most commonly used character code is the International Reference Alphabet (IRA)
 - Referred to in the United States as the American Standard Code for Information Interchange (ASCII)
- Another code used to encode characters is the Extended Binary Coded Decimal Interchange Code (EBCDIC)
 - EBCDIC is used on IBM mainframes

Logical Data

- An n -bit unit consisting of n 1-bit items of data, each item having the value 0 or 1
- Two advantages to bit-oriented view:
 - Memory can be used most efficiently for storing an array of Boolean or binary data items in which each item can take on only the values 1 (true) and 0 (false)
 - To manipulate the bits of a data item
 - If floating-point operations are implemented in software, we need to be able to shift significant bits in some operations
 - To convert from IRA to packed decimal, we need to extract the rightmost 4 bits of each byte

x86 and ARM Data Types

- Can refer to the example of x86 and ARM data types in the following slides:
- [x86](#)
- [ARM](#)

Types of Operations

Data transfer	Transfer data from one location to another
	If memory is involved: Determine memory address Perform virtual-to-actual-memory address transformation Check cache Initiate memory read/write
Arithmetic	May involve data transfer, before and/or after
	Perform function in ALU
	Set condition codes and flags
Logical	Same as arithmetic
Conversion	Similar to arithmetic and logical. May involve special logic to perform conversion
Transfer of control	Update program counter. For subroutine call/return, manage parameter passing and linkage
I/O	Issue command to I/O module
	If memory-mapped I/O, determine memory-mapped address

Data Transfer

Most fundamental type of machine instruction

Must specify:

- Location of the source and destination operands
- The length of data to be transferred must be indicated
- The mode of addressing for each operand must be specified

Operation Name	Description
MOV Dest, Source	Move data between registers or between register and memory or immediate to register.
XCHG Op1, Op2	Swap contents between two registers or register and memory.
PUSH Source	Decrements stack pointer (ESP register), then copies the source operand to the top of stack.
POP Dest	Copies top of stack to destination and increments ESP.

Table 13.5

Examples of IBM EAS/390 Data Transfer Operations

Operation Mnemonic	Name	Number of Bits Transferred	Description
L	Load	32	Transfer from memory to register
LH	Load Halfword	16	Transfer from memory to register
LR	Load	32	Transfer from register to register
LER	Load (short)	32	Transfer from floating-point register to floating-point register
LE	Load (short)	32	Transfer from memory to floating-point register
LDR	Load (long)	64	Transfer from floating-point register to floating-point register
LD	Load (long)	64	Transfer from memory to floating-point register
ST	Store	32	Transfer from register to memory
STH	Store Halfword	16	Transfer from register to memory
STC	Store Character	8	Transfer from register to memory
STE	Store (short)	32	Transfer from floating-point register to memory
STD	Store (long)	64	Transfer from floating-point register to memory

(Table can be found on page 448 in the textbook.)

Arithmetic

- Most machines provide the basic arithmetic operations of add, subtract, multiply, and divide
- These are provided for signed integer (fixed-point) numbers
- Often they are also provided for floating-point and packed decimal numbers
- Other possible operations include a variety of single-operand instructions:
 - Absolute
 - Take the absolute value of the operand
 - Negate
 - Negate the operand
 - Increment
 - Add 1 to the operand
 - Decrement
 - Subtract 1 from the operand

Operation Name	Description
ADD Dest, Source	Adds the destination and the source operand and stores the result in the destination. Destination can be register or memory. Source can be register, memory, or immediate.
SUB Dest, Source	Subtracts the source from the destination and stores the result in the destination.
MUL Op	Unsigned integer multiplication of the operand by the AL, AX, or EAX register and stores in the register. Opcode indicates size of register.
IMUL Op	Signed integer multiplication.
DIV Op	Divides unsigned the value in the AX, DX:AX, EDX:EAX, or RDX:RAX registers (dividend) by the source operand (divisor) and stores the result in the AX (AH:AL), DX:AX, EDX:EAX, or RDX:RAX registers.
IDIV Op	Signed integer division.
INC Op	Adds 1 to the destination operand, while preserving the state of the CF flag.
DEC Op	Subtracts 1 from the destination operand, while preserving the state of the CF flag.
NEG Op	Replaces the value of operand with (0 – operand), using twos complement representation.
CMP Op1, Op2	Compares the two operands by subtracting the second operand from the first operand and sets the status flags in the EFLAGS register according to the results.

[click on this picture to link to the x86 instructions list](#)

Table 13.6

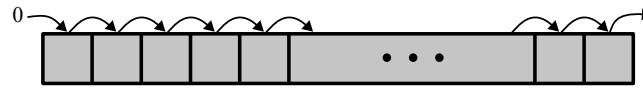
Basic Logical Operations

P	Q	NOT P	P AND Q	P OR Q	P XOR Q	P = Q
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

[Link to the x86 instructions list](#)

Figure 13.6

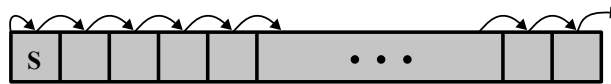
Shift and Rotate Operations



(a) Logical right shift



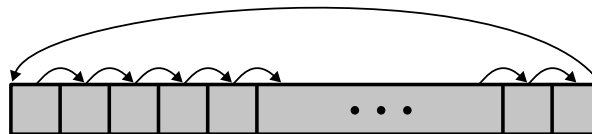
(b) Logical left shift



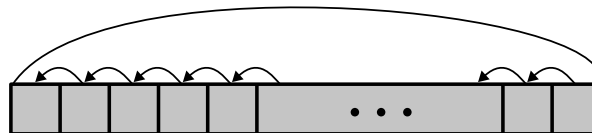
(c) Arithmetic right shift



(d) Arithmetic left shift



(e) Right rotate



(f) Left rotate

Table 13.7

Examples of Shift and Rotate Operations

Input	Operation	Result
10100110	Logical right shift (3 bits)	00010100
10100110	Logical left shift (3 bits)	00110000
10100110	Arithmetic right shift (3 bits)	11110100
10100110	Arithmetic left shift (3 bits)	10110000
10100110	Right rotate (3 bits)	11010100
10100110	Left rotate (3 bits)	00110101

Conversion

Instructions that
change the
format or
operate on the
format of data

An example
is converting
from
decimal to
binary

An example of a
more complex
editing
instruction is the
EAS/390
Translate (TR)
instruction

Input/Output

- Variety of approaches taken:
 - Isolated programmed I/O
 - Memory-mapped programmed I/O
 - DMA
 - Use of an I/O processor
- Many implementations provide only a few I/O instructions, with the specific actions specified by parameters, codes, or command words

System Control

Instructions that can be executed only while the processor is in a certain privileged state or is executing a program in a special privileged area of memory

Typically these instructions are reserved for the use of the operating system

Examples of system control operations:

A system control instruction may read or alter a control register

An instruction to read or modify a storage protection key

Access to process control blocks in a multiprogramming system

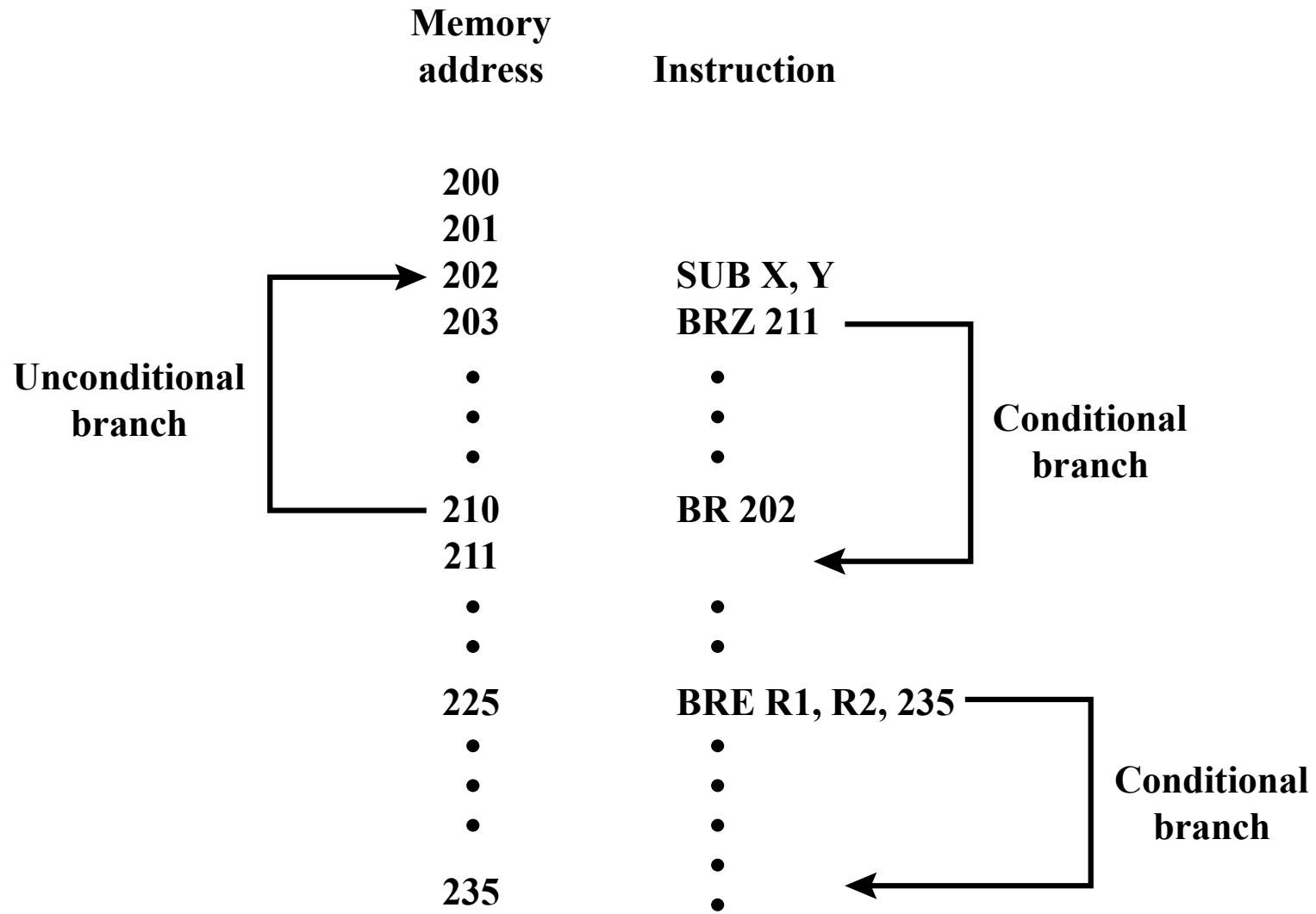
Transfer of Control

- Reasons why transfer-of-control operations are required:
 - It is essential to be able to execute each instruction more than once
 - Virtually all programs involve some decision making
 - It helps if there are mechanisms for breaking the task up into smaller pieces that can be worked on one at a time
- Most common transfer-of-control operations found in instruction sets:
 - Branch
 - Skip
 - Procedure call

[Link to the x86 instructions list](#)

Figure 13.7

Branch Instructions

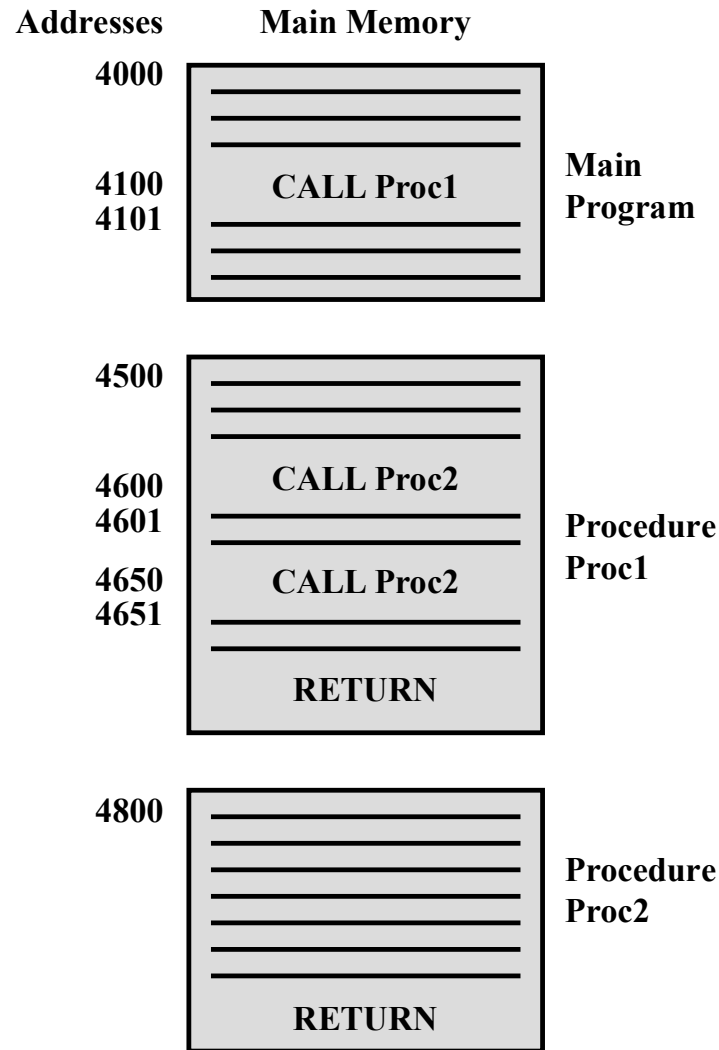


Procedure Call Instructions

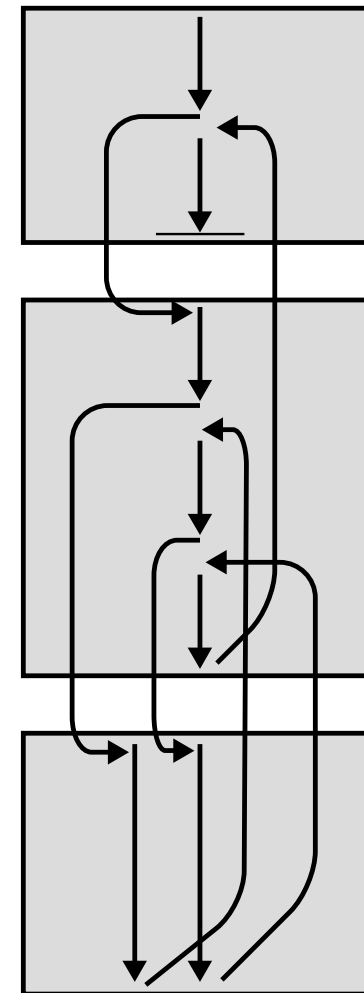
- Self-contained computer program that is incorporated into a larger program
 - At any point in the program the procedure may be invoked, or *called*
 - Processor is instructed to go and execute the entire procedure and then return to the point from which the call took place
- Two principal reasons for use of procedures:
 - Economy
 - A procedure allows the same piece of code to be used many times
 - Modularity
- Involves two basic instructions:
 - A call instruction that branches from the present location to the procedure
 - Return instruction that returns from the procedure to the place from which it was called

Figure 13.8

Nested Procedures



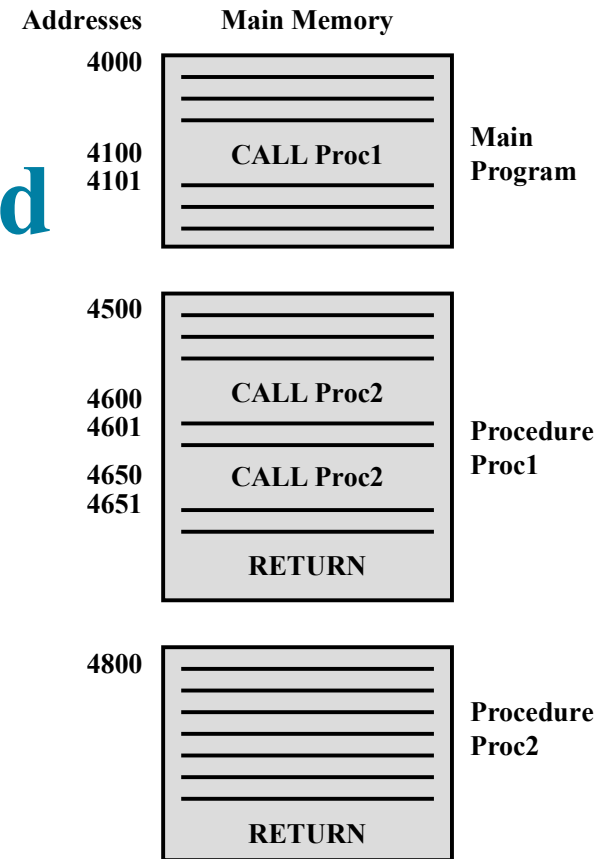
(a) Calls and returns



(b) Execution sequence

Figure 13.9

Use of Stack to Implement Nested Subroutines of Figure 13.8



(a) Calls and returns

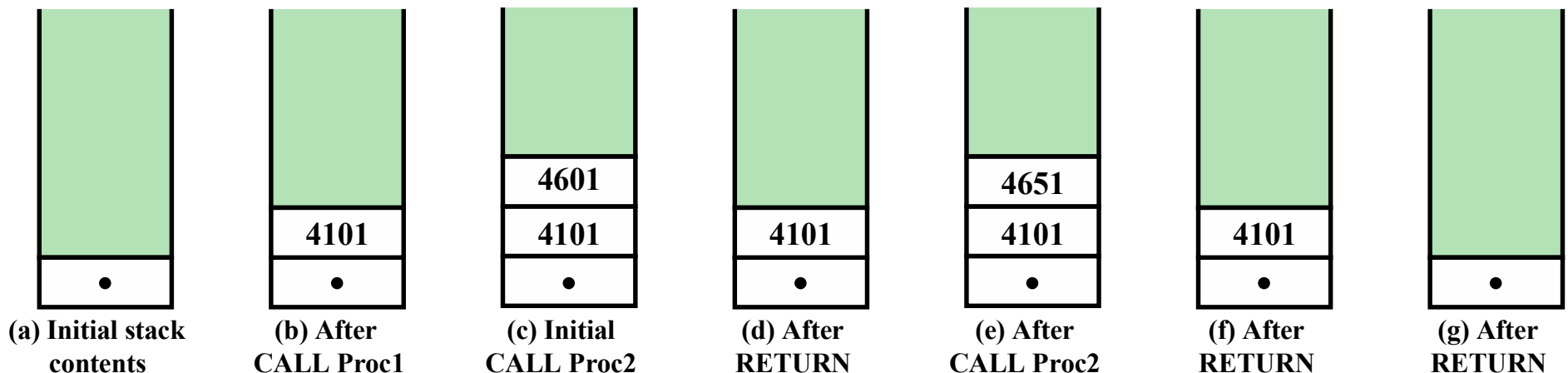
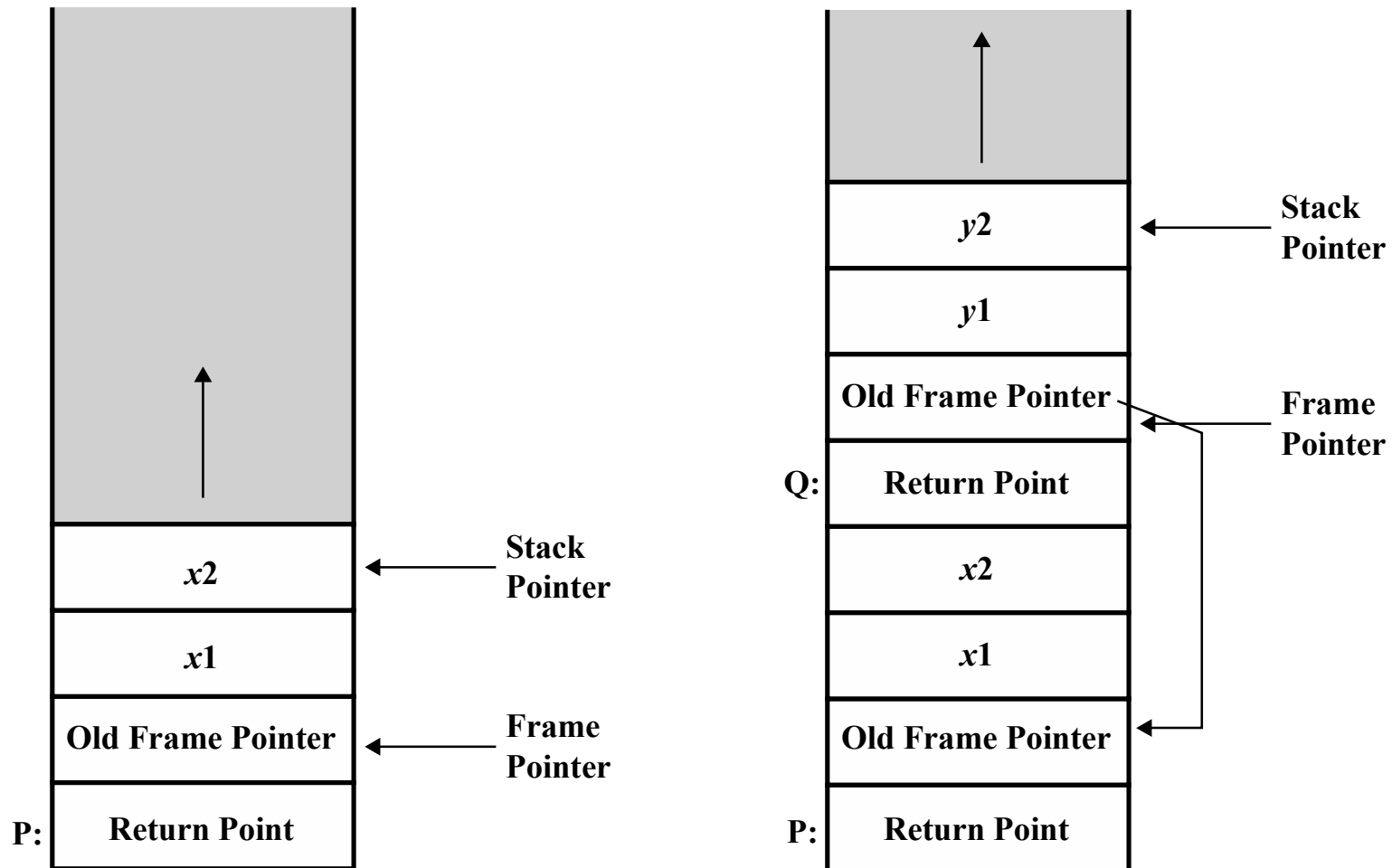


Figure 13.10

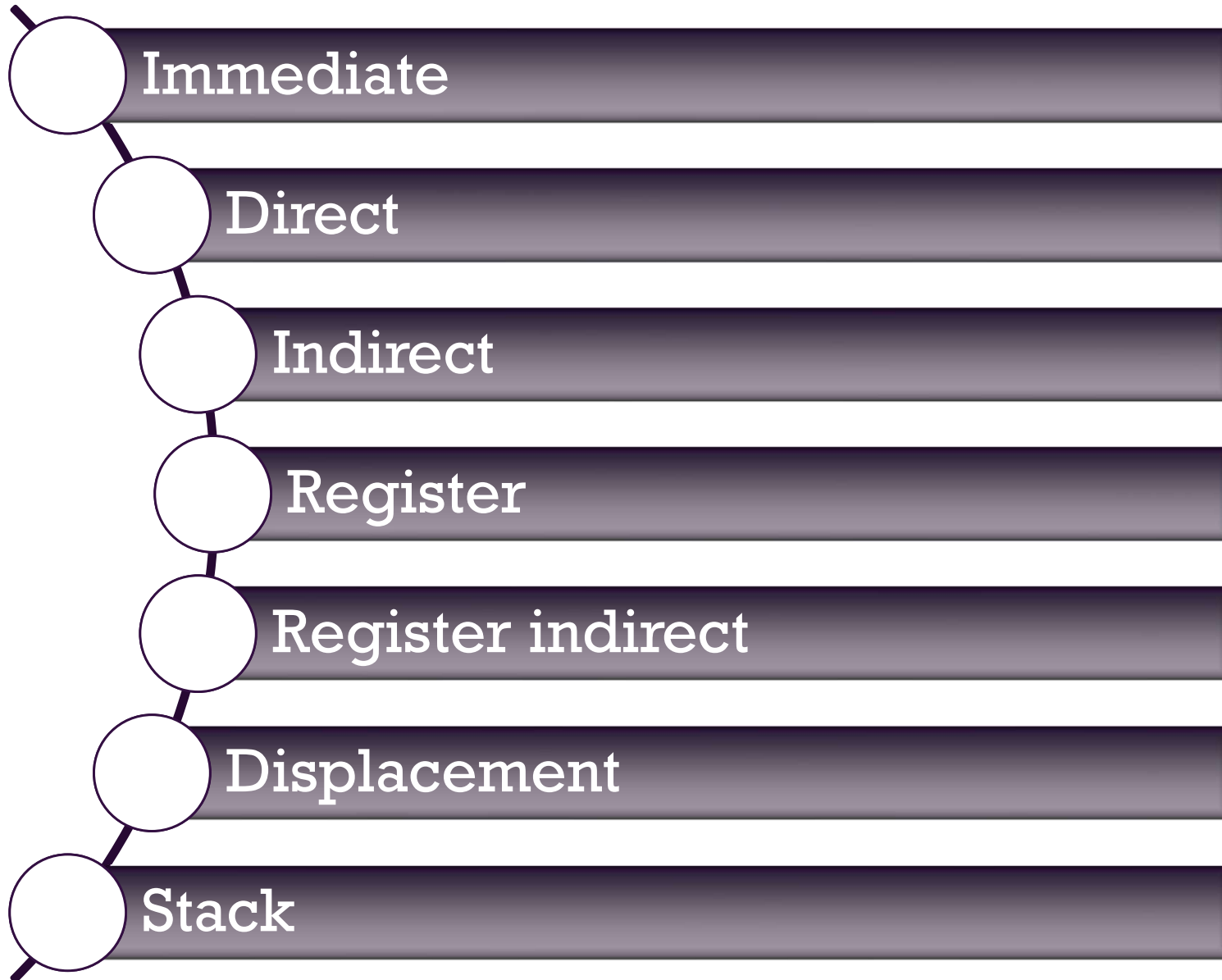
Stack Frame Growth Using Sample Procedures P and Q



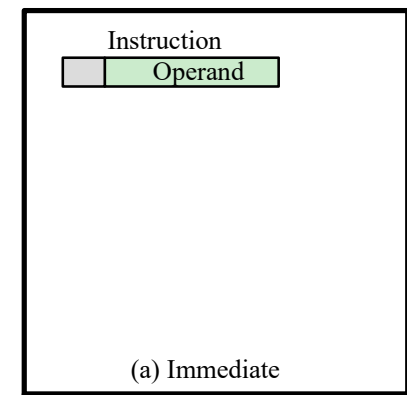
x86 and ARM Operation Types

- Can refer to the example of x86 and ARM operation types in the following slides:
- x86
- ARM

Addressing Modes



Immediate Addressing



- Simplest form of addressing
- Operand = A
- This mode can be used to define and use constants or set initial values of variables
 - Typically the number will be stored in twos complement form
 - The leftmost bit of the operand field is used as a sign bit
- Advantage:
 - No memory reference other than the instruction fetch is required to obtain the operand, thus saving one memory or cache cycle in the instruction cycle
- Disadvantage:
 - The size of the number is restricted to the size of the address field, which, in most instruction sets, is small compared with the word length

Direct Addressing

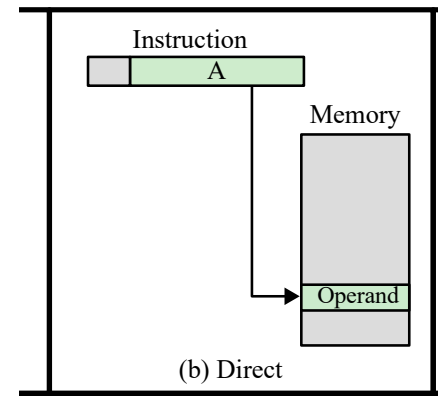
Address field contains the effective address of the operand

Effective address (EA) = address field (A)

Was common in earlier generations of computers

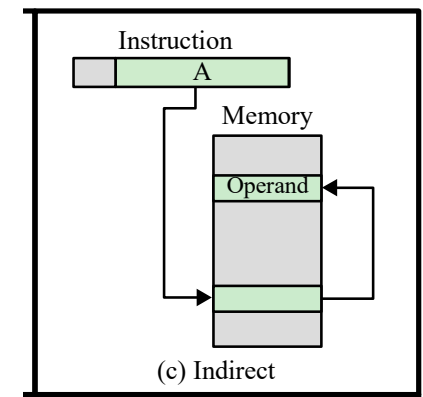
Requires only one memory reference and no special calculation

Limitation is that it provides only a limited address space

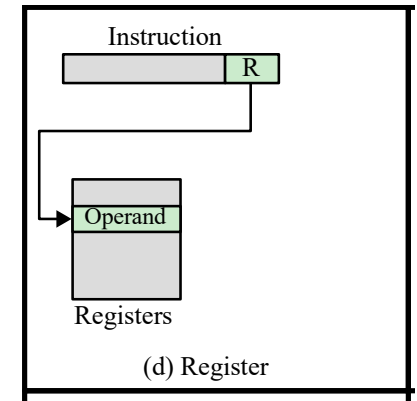
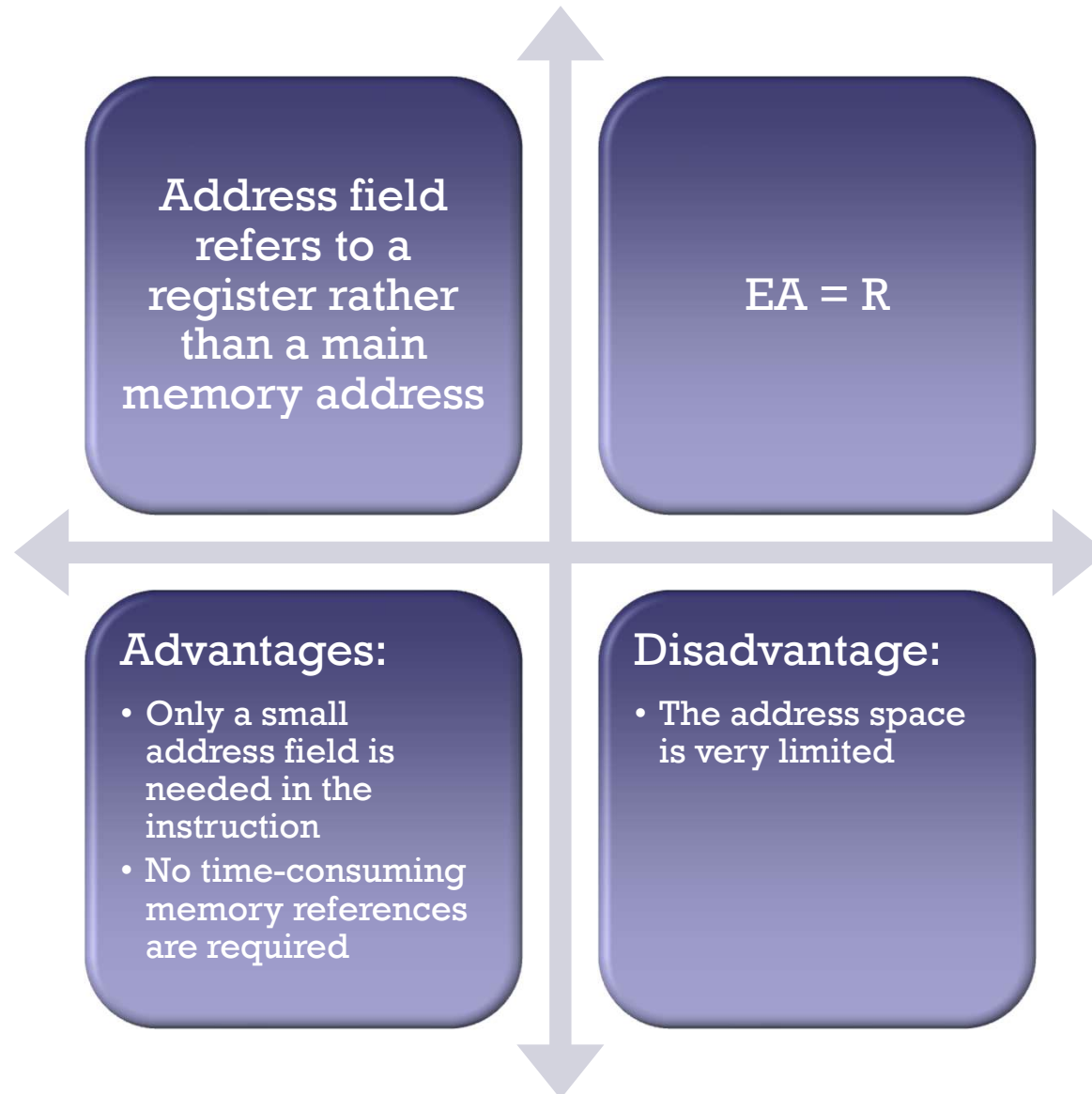


Indirect Addressing

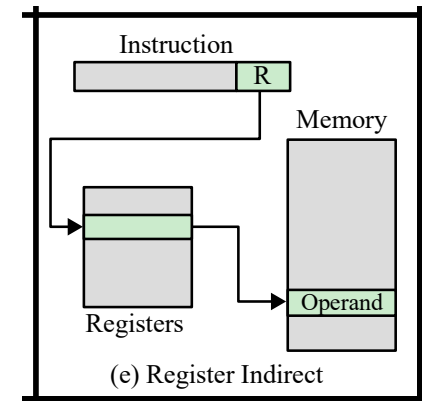
- Reference to the address of a word in memory which contains a full-length address of the operand
- $EA = (A)$
 - Parentheses are to be interpreted as meaning *contents of*
- Advantage:
 - For a word length of N an address space of 2^N is now available
- Disadvantage:
 - Instruction execution requires two memory references to fetch the operand
 - One to get its address and a second to get its value
 - A rarely used variant of indirect addressing is multilevel or cascaded indirect addressing
 - $EA = (\dots (A) \dots)$
 - Disadvantage is that three or more memory references could be required to fetch an operand



Register Addressing



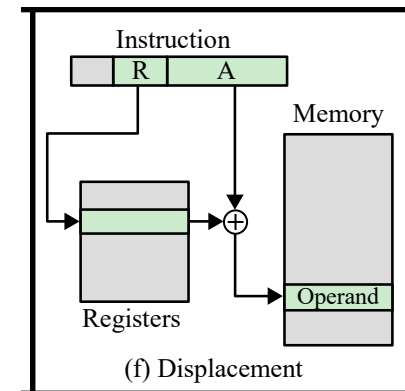
Register Indirect Addressing



- Analogous to indirect addressing
 - The only difference is whether the address field refers to a memory location or a register
- $EA = (R)$
- Address space limitation of the address field is overcome by having that field refer to a word-length location containing an address
- Uses one less memory reference than indirect addressing

Displacement Addressing

- Combines the capabilities of direct addressing and register indirect addressing
- $EA = A + (R)$
- Requires that the instruction have two address fields, at least one of which is explicit
 - The value contained in one address field (value = A) is used directly
 - The other address field refers to a register whose contents are added to A to produce the effective address
- Most common uses:
 - Relative addressing
 - Base-register addressing
 - Indexing



Relative Addressing

The implicitly referenced register is the program counter (PC)

- The next instruction address is added to the address field to produce the EA
- Typically the address field is treated as a twos complement number for this operation
- Thus the effective address is a displacement relative to the address of the instruction

Exploits the concept of locality

Saves address bits in the instruction if most memory references are relatively near to the instruction being executed

Base-Register Addressing

- The referenced register contains a main memory address and the address field contains a displacement from that address
- The register reference may be explicit or implicit
- Exploits the locality of memory references
- Convenient means of implementing segmentation
- In some implementations a single segment base register is employed and is used implicitly
- In others the programmer may choose a register to hold the base address of a segment and the instruction must reference it explicitly

Indexing

- The address field references a main memory address and the referenced register contains a positive displacement from that address
- The method of calculating the EA is the same as for base-register addressing
- An important use is to provide an efficient mechanism for performing iterative operations
- Autoindexing
 - Automatically increment or decrement the index register after each reference to it
 - $EA = A + (R)$
 - $(R) = (R) + 1$
- Postindexing
 - Indexing is performed after the indirection
 - $EA = (A) + (R)$
- Preindexing
 - Indexing is performed before the indirection
 - $EA = (A + (R))$

Stack Addressing

- A stack is a linear array of locations
 - Sometimes referred to as a *pushdown list* or *last-in-first-out queue*
- A stack is a reserved block of locations
 - Items are appended to the top of the stack so that the block is partially filled
- Associated with the stack is a pointer whose value is the address of the top of the stack
 - The stack pointer is maintained in a register
 - Thus references to stack locations in memory are in fact register indirect addresses
- Is a form of implied addressing
- The machine instructions need not include a memory reference but implicitly operate on the top of the stack

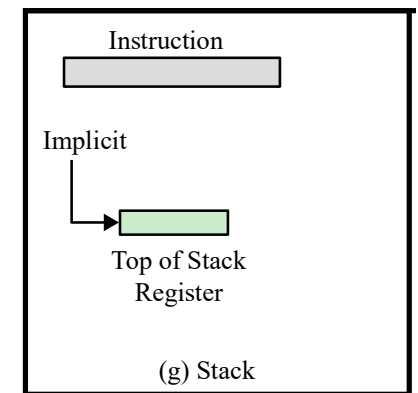


Figure 14.1

Addressing Modes

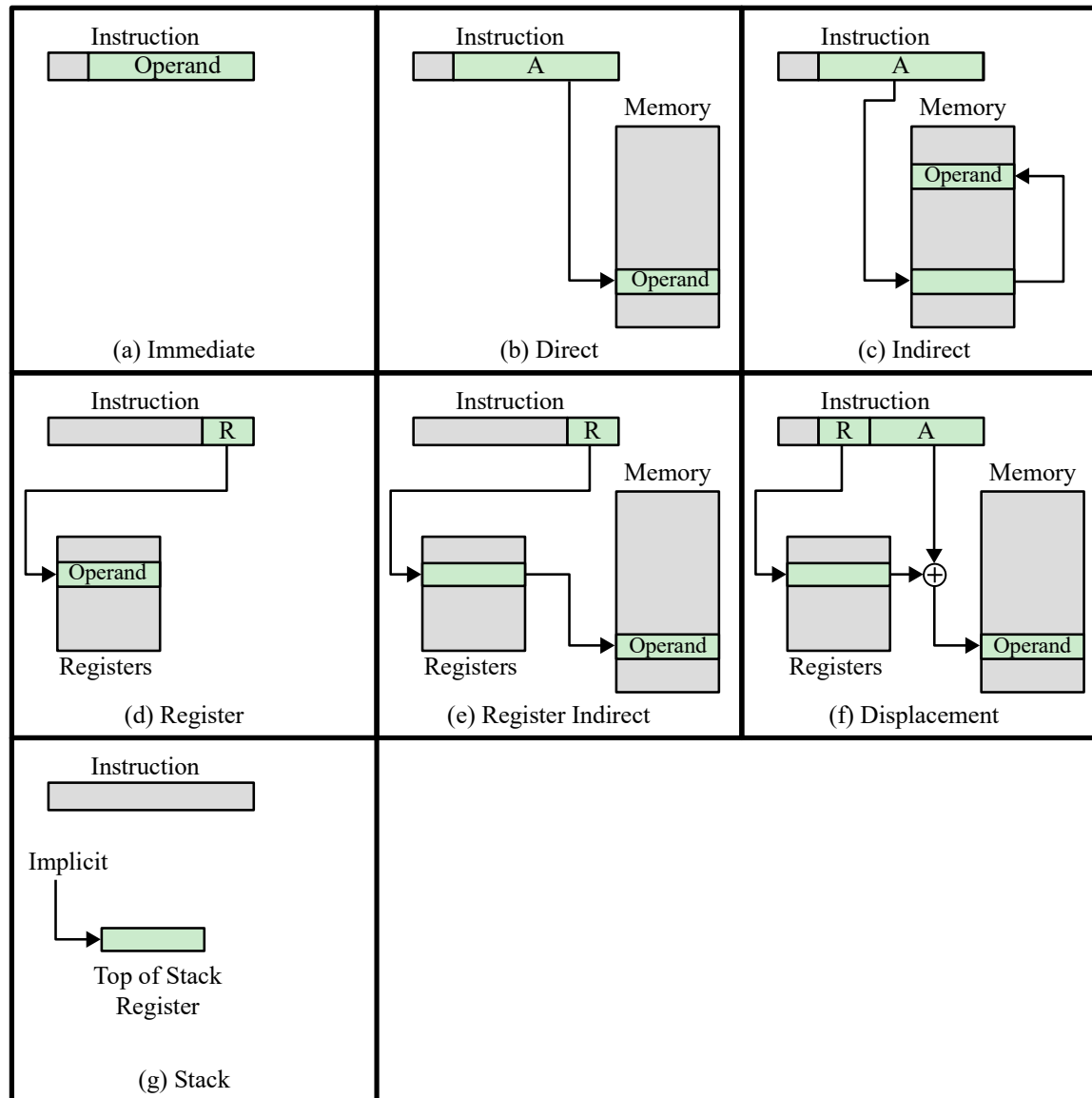


Table 14.1

Basic Addressing Modes

Mode	Algorithm	Principal Advantage	Principal Disadvantage
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory reference	Limited address space
Register indirect	EA = (R)	Large address space	Extra memory reference
Displacement	EA = A + (R)	Flexibility	Complexity
Stack	EA = top of stack	No memory reference	Limited applicability

x86 and ARM Addressing Mode

- Can refer to the example of x86 and ARM addressing mode in the following slides:
- [x86](#)
- [ARM](#)

Instruction Formats

Define the layout of the bits of an instruction, in terms of its constituent fields

Must include an opcode and, implicitly or explicitly, indicate the addressing mode for each operand

For most instruction sets more than one instruction format is used

Instruction Length

- Most basic design issue
- Affects, and is affected by:
 - Memory size
 - Memory organization
 - Bus structure
 - Processor complexity
 - Processor speed
- Should be equal to the memory-transfer length or one should be a multiple of the other
- Should be a multiple of the character length, which is usually 8 bits, and of the length of fixed-point numbers

Allocation of Bits

Number of
addressing
modes

Number of
operands

Register
versus
memory

Number of
register sets

Address
range

Address
granularity

PDP, VAX, x86 and ARM

Instruction Formats

- Can refer to the example of x86 and ARM addressing mode in the following slides:
- PDP
- VAX
- x86
- ARM

Additional Notes on x86 and ARM

Table 13.2

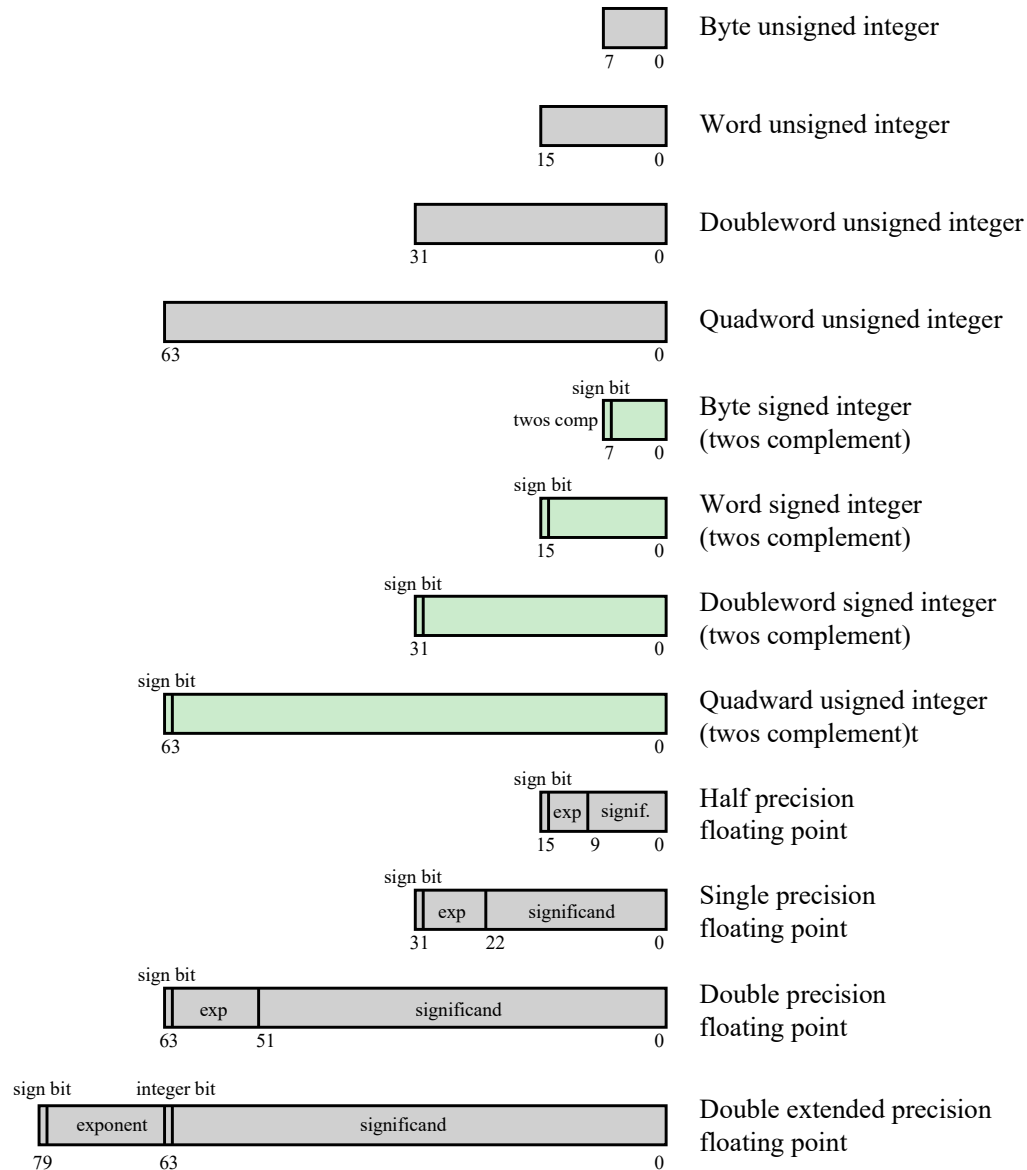
x86 Data Types

Data Type	Description
General	Byte, word (16 bits), doubleword (32 bits), quadword (64 bits), and double quadword (128 bits) locations with arbitrary binary contents.
Integer	A signed binary value contained in a byte, word, or doubleword, using twos complement representation.
Ordinal	An unsigned integer contained in a byte, word, or doubleword.
Unpacked binary coded decimal (BCD)	A representation of a BCD digit in the range 0 through 9, with one digit in each byte.
Packed BCD	Packed byte representation of two BCD digits; value in the range 0 to 99.
Near pointer	A 16-bit, 32-bit, or 64-bit effective address that represents the offset within a segment. Used for all pointers in a nonsegmented memory and for references within a segment in a segmented memory.
Far pointer	A logical address consisting of a 16-bit segment selector and an offset of 16, 32, or 64 bits. Far pointers are used for memory references in a segmented memory model where the identity of a segment being accessed must be specified explicitly.
Bit field	A contiguous sequence of bits in which the position of each bit is considered as an independent unit. A bit string can begin at any bit position of any byte and can contain up to 32 bits.
Bit string	A contiguous sequence of bits, containing from zero to $2^{23} - 1$ bits.
Byte string	A contiguous sequence of bytes, words, or doublewords, containing from zero to $2^{23} - 1$ bytes.
Floating point	See Figure 13.4.
Packed SIMD (single instruction, multiple data)	Packed 64-bit and 128-bit data types.

(Table can be found on page 443
In the textbook)

Figure 13.4

x86 Numeric Data Formats



Single-Instruction-Multiple-Data (SIMD)

Data Types

- Introduced to the x86 architecture as part of the extensions of the instruction set to optimize performance of multimedia applications
- These extensions include MMX (multimedia extensions) and SSE (streaming SIMD extensions)
- Data types:
 - Packed byte and packed byte integer
 - Packed word and packed word integer
 - Packed doubleword and packed doubleword integer
 - Packed quadword and packed quadword integer
 - Packed single-precision floating-point and packed double-precision floating-point

[BACK](#)

ARM Data Types

ARM processors support data types of:

- 8 (byte)
- 16 (halfword)
- 32 (word) bits in length

All three data types can also be used for two's complement signed integers

For all three data types an unsigned interpretation is supported in which the value represents an unsigned, nonnegative integer

Alignment checking

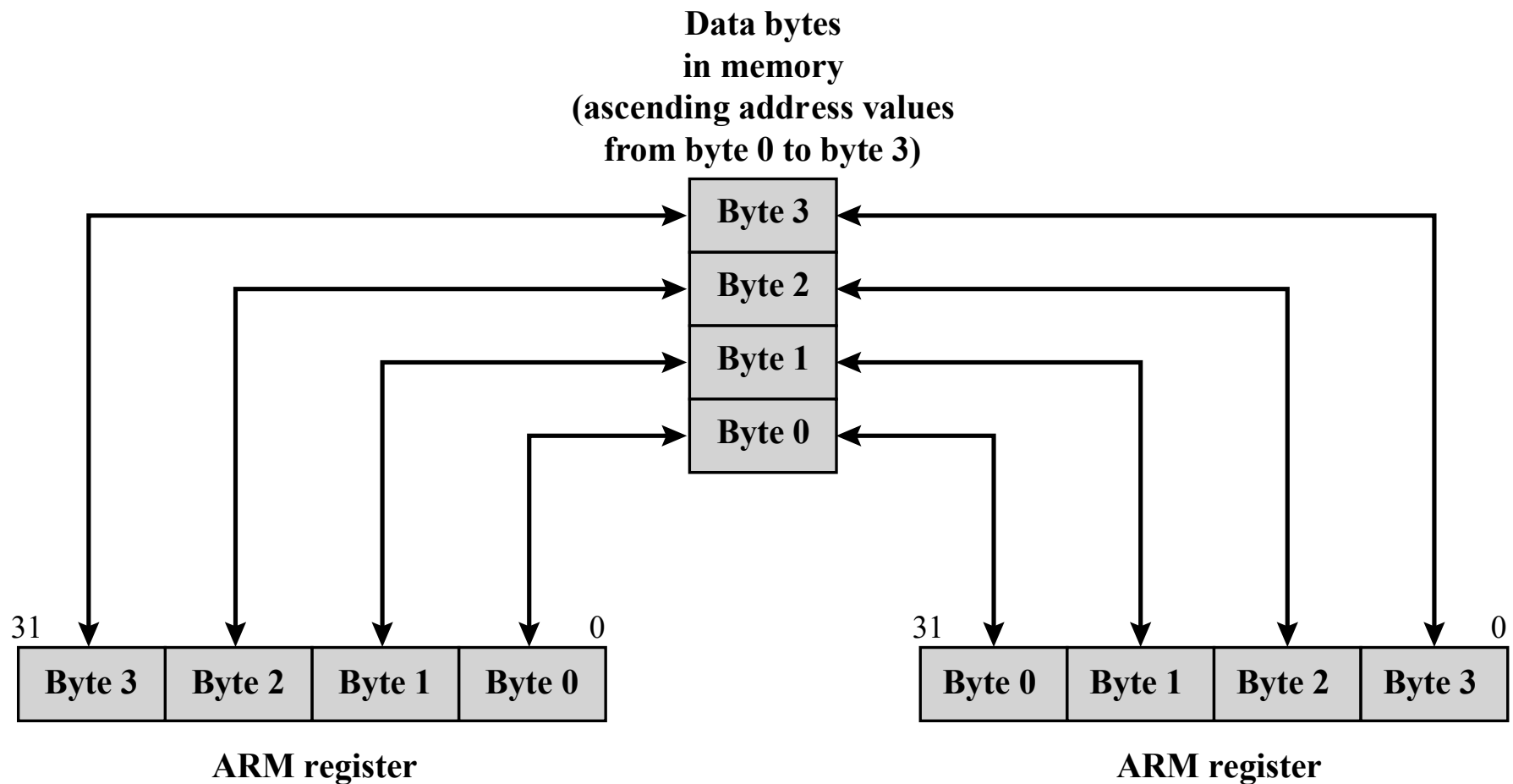
- When the appropriate control bit is set, a data abort signal indicates an alignment fault for attempting unaligned access

Unaligned access

- When this option is enabled, the processor uses one or more memory accesses to generate the required transfer of adjacent bytes transparently to the programmer

Figure 13.5

ARM Endian Support—Word Load/Store with E-Bit



program status register E-bit = 0

program status register E-bit = 1

(a) Data Transfer

Table 13.3 Common x86 Instruction Set Operations (1 of 3)

Operation Name	Description
MOV Dest, Source	Move data between registers or between register and memory or immediate to register.
XCHG Op1, Op2	Swap contents between two registers or register and memory.
PUSH Source	Decrements stack pointer (ESP register), then copies the source operand to the top of stack.
POP Dest	Copies top of stack to destination and increments ESP.

(b) Arithmetic

Operation Name	Description
ADD Dest, Source	Adds the destination and the source operand and stores the result in the destination. Destination can be register or memory. Source can be register, memory, or immediate.
SUB Dest, Source	Subtracts the source from the destination and stores the result in the destination.
MUL Op	Unsigned integer multiplication of the operand by the AL, AX, or EAX register and stores in the register. Opcode indicates size of register.
IMUL Op	Signed integer multiplication.
DIV Op	Divides unsigned the value in the AX, DX:AX, EDX:EAX, or RDX:RAX registers (dividend) by the source operand (divisor) and stores the result in the AX (AH:AL), DX:AX, EDX:EAX, or RDX:RAX registers.
IDIV Op	Signed integer division.
INC Op	Adds 1 to the destination operand, while preserving the state of the CF flag.
DEC Op	Subtracts 1 from the destination operand, while preserving the state of the CF flag.
NEG Op	Replaces the value of operand with (0 – operand), using twos complement representation.
CMP Op1, Op2	Compares the two operands by subtracting the second operand from the first operand and sets the status flags in the EFLAGS register according to the results.

(Table can be found on page 446-447 in the textbook.)

(c) Shift and Rotate

Table 13.3 Common x86 Instruction Set Operations (2 of 3)

Operation Name	Description
SAL Op, Quantity	Shifts the source operand left by from 1 to 31 bit positions. Empty bit positions are cleared. The CF flag is loaded with the last bit shifted out of the operand.
SAR Op, Quantity	Shifts the source operand right by from 1 to 31 bit positions. Empty bit positions are cleared if the operand is positive and set if the operand is negative. The CF flag is loaded with the last bit shifted out of the operand.
SHR Op, Quantity	Shifts the source operand right by from 1 to 31 bit positions. Empty bit positions are cleared and the CF flag is loaded with the last bit shifted out of the operand.
ROL Op, Quantity	Rotate bits to the left, with wraparound. The CF flag is loaded with the last bit shifted out of the operand.
ROR Op, Quantity	Rotate bits to the right, with wraparound. The CF flag is loaded with the last bit shifted out of the operand.
RCL Op, Quantity	Rotate bits to the left, including the CF flag, with wraparound. This instruction treats the CF flag as a one-bit extension on the upper end of the operand.
RCR Op, Quantity	Rotate bits to the right, including the CF flag, with wraparound. This instruction treats the CF flag as a one-bit extension on the lower end of the operand.

(d) Logical

Operation Name	Description
NOT Op	Inverts each bit of the operand.
AND Dest, Source	Performs a bitwise AND operation on the destination and source operands and stores the result in the destination operand.
OR Dest, Source	Performs a bitwise OR operation on the destination and source operands and stores the result in the destination operand.
XOR Dest, Source	Performs a bitwise XOR operation on the destination and source operands and stores the result in the destination operand.
TEST Op1, Op2	Performs a bitwise AND operation on the two operands and sets the S, Z, and P status flags. The operands are unchanged.

(Table can be found on page 446-447 in the textbook.)

(e) Transfer of Control

Table 13.3 Common x86 Instruction Set Operations (3 of 3)

Operation Name	Description
CALL proc	Saves procedure linking information on the stack and branches to the called procedure specified using the operand. The operand specifies the address of the first instruction in the called procedure.
RET	Transfers program control to a return address located on the top of the stack. The return is made to the instruction that follows the CALL instruction.
JMP Dest	Transfers program control to a different point in the instruction stream without recording return information. The operand specifies the address of the instruction being jumped to.
Jcc Dest	Checks the state of one or more of the status flags in the EFLAGS register (CF, OF, PF, SF, and ZF) and, if the flags are in the specified state (condition), performs a jump to the target instruction specified by the destination operand. See Tables 13.8 and 13.9.
NOP	This instruction performs no operation. It is a one-byte or multi-byte NOP that takes up space in the instruction stream but does not impact machine context, except for the EIP register.
HLT	Stops instruction execution and places the processor in a HALT state. An enabled interrupt, a debug exception, the BINIT# signal, the INIT# signal, or the RESET# signal will resume execution.
WAIT	Causes the processor to repeatedly check for and handle pending, unmasked, floating-point exceptions before proceeding.
INT Nr	Interrupts current program, runs specified interrupt program

(f) Input/Output

Operation Name	Description
IN Dest, Source	Copies the data from the I/O port specified by the source operand to the destination operand, which is a register location.
INS Dest, Source	Copies the data from the I/O port specified by the source operand to the destination operand, which is a memory location.
OUT Dest, Source	Copies the byte, word, or doubleword value from the source register to the I/O port specified by the destination operand.
XOR Dest, Source	Copies byte, word, or doubleword from the source operand to the I/O port specified with the destination operand. The source operand is a memory location.

(Table can be found on page 446-447 in the textbook.)

x86 Operation Types

- The x86 provides a complex array of operation types including a number of specialized instructions
- The intent was to provide tools for the compiler writer to produce optimized machine language translation of high-level language programs
- Provides four instructions to support procedure call/return:
 - CALL
 - ENTER
 - LEAVE
 - RETURN
- When a new procedure is called the following must be performed upon entry to the new procedure:
 - Push the return point on the stack
 - Push the current frame pointer on the stack
 - Copy the stack pointer as the new value of the frame pointer
 - Adjust the stack pointer to allocate a frame

Table 13.8

x86 Status Flags

Status Bit	Name	Description
C	Carry	Indicates carrying or borrowing out of the left-most bit position following an arithmetic operation. Also modified by some of the shift and rotate operations.
P	Parity	Parity of the least-significant byte of the result of an arithmetic or logic operation. 1 indicates even parity; 0 indicates odd parity.
A	Auxiliary Carry	Represents carrying or borrowing between half-bytes of an 8-bit arithmetic or logic operation. Used in binary-coded decimal arithmetic.
Z	Zero	Indicates that the result of an arithmetic or logic operation is 0.
S	Sign	Indicates the sign of the result of an arithmetic or logic operation.
O	Overflow	Indicates an arithmetic overflow after an addition or subtraction for twos complement arithmetic.

Table 13.9 x86 Condition Codes for Conditional Jump and SETcc Instructions

Symbol	Condition Tested	Comment
A, NBE	$C = 0 \text{ AND } Z = 0$	Above; Not below or equal (greater than, unsigned)
AE, NB, NC	$C = 0$	Above or equal; Not below (greater than or equal, unsigned); Not carry
B, NAE, C	$C = 1$	Below; Not above or equal (less than, unsigned); Carry set
BE, NA	$C = 1 \text{ OR } Z = 1$	Below or equal; Not above (less than or equal, unsigned)
E, Z	$Z = 1$	Equal; Zero (signed or unsigned)
G, NLE	$[(S = 1 \text{ AND } O = 1) \text{ OR } (S = 0 \text{ AND } O = 0)] \text{ AND } [Z = 0]$	Greater than; Not less than or equal (signed)
GE, NL	$(S = 1 \text{ AND } O = 1) \text{ OR } (S = 0 \text{ AND } O = 0)$	Greater than or equal; Not less than (signed)
L, NGE	$(S = 1 \text{ AND } O = 0) \text{ OR } (S = 0 \text{ AND } O = 0)$	Less than; Not greater than or equal (signed)
LE, NG	$(S = 1 \text{ AND } O = 0) \text{ OR } (S = 0 \text{ AND } O = 1) \text{ OR } (Z = 1)$	Less than or equal; Not greater than (signed)
NE, NZ	$Z = 0$	Not equal; Not zero (signed or unsigned)
NO	$O = 0$	No overflow
NS	$S = 0$	Not sign (not negative)
NP, PO	$P = 0$	Not parity; Parity odd
O	$O = 1$	Overflow
P	$P = 1$	Parity; Parity even
S	$S = 1$	Sign (negative)

(Table can be found on page 460 in the textbook.)

x86 Single-Instruction, Multiple-Data (SIMD) Instructions

- 1996 Intel introduced MMX technology into its Pentium product line
 - MMX is a set of highly optimized instructions for multimedia tasks
- Video and audio data are typically composed of large arrays of small data types
- Three new data types are defined in MMX
 - Packed byte
 - Packed word
 - Packed doubleword
- Each data type is 64 bits in length and consists of multiple smaller data fields, each of which holds a fixed-point integer

Table 13.10

MMX

Instruction Set

Category	Instruction	Description
Arithmetic	PADD [B, W, D]	Parallel add of packed eight bytes, four 16-bit words, or two 32-bit doublewords, with wraparound.
	PADD [B, W]	Add with saturation.
	PADDUS [B, W]	Add unsigned with saturation.
	PSUB [B, W, D]	Subtract with wraparound.
	PSUBS [B, W]	Subtract with saturation.
	PSUBUS [B, W]	Subtract unsigned with saturation.
	PMULHW	Parallel multiply of four signed 16-bit words, with high-order 16 bits of 32-bit result chosen.
	PMULLW	Parallel multiply of four signed 16-bit words, with low-order 16 bits of 32-bit result chosen.
	PMADDWD	Parallel multiply of four signed 16-bit words; add together adjacent pairs of 32-bit results.
Comparison	PCMPEQ [B, W, D]	Parallel compare for equality; result is mask of 1s if true or 0s if false.
	PCMPGT [B, W, D]	Parallel compare for greater than; result is mask of 1s if true or 0s if false.
Conversion	PACKUSWB	Pack words into bytes with unsigned saturation.
	PACKSS [WB, DW]	Pack words into bytes, or doublewords into words, with signed saturation.
	PUNPCKH [BW, WD, DQ]	Parallel unpack (interleaved merge) high-order bytes, words, or doublewords from MMX register.
	PUNPCKL [BW, WD, DQ]	Parallel unpack (interleaved merge) low-order bytes, words, or doublewords from MMX register.
Logical	PAND	64-bit bitwise logical AND
	PANDN	64-bit bitwise logical AND NOT
	POR	64-bit bitwise logical OR
	PXOR	64-bit bitwise logical XOR
Shift	PSLL [W, D, Q]	Parallel logical left shift of packed words, doublewords, or quadword by amount specified in MMX register or immediate value.
	PSRL [W, D, Q]	Parallel logical right shift of packed words, doublewords, or quadword.
	PSRA [W, D]	Parallel arithmetic right shift of packed words, doublewords, or quadword.
Data transfer	MOV [D, Q]	Move doubleword or quadword to/from MMX register.
Statemgt	EMMS	Empty MMX state (empty FP registers tag bits).

Note: If an instruction supports multiple data types [byte (B), word (W), doubleword (D), quadword (Q)], the data types are indicated in brackets.

(Table can be found on page 462 in the textbook.)

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 13.11

Image Compositing on Color Plane Representation

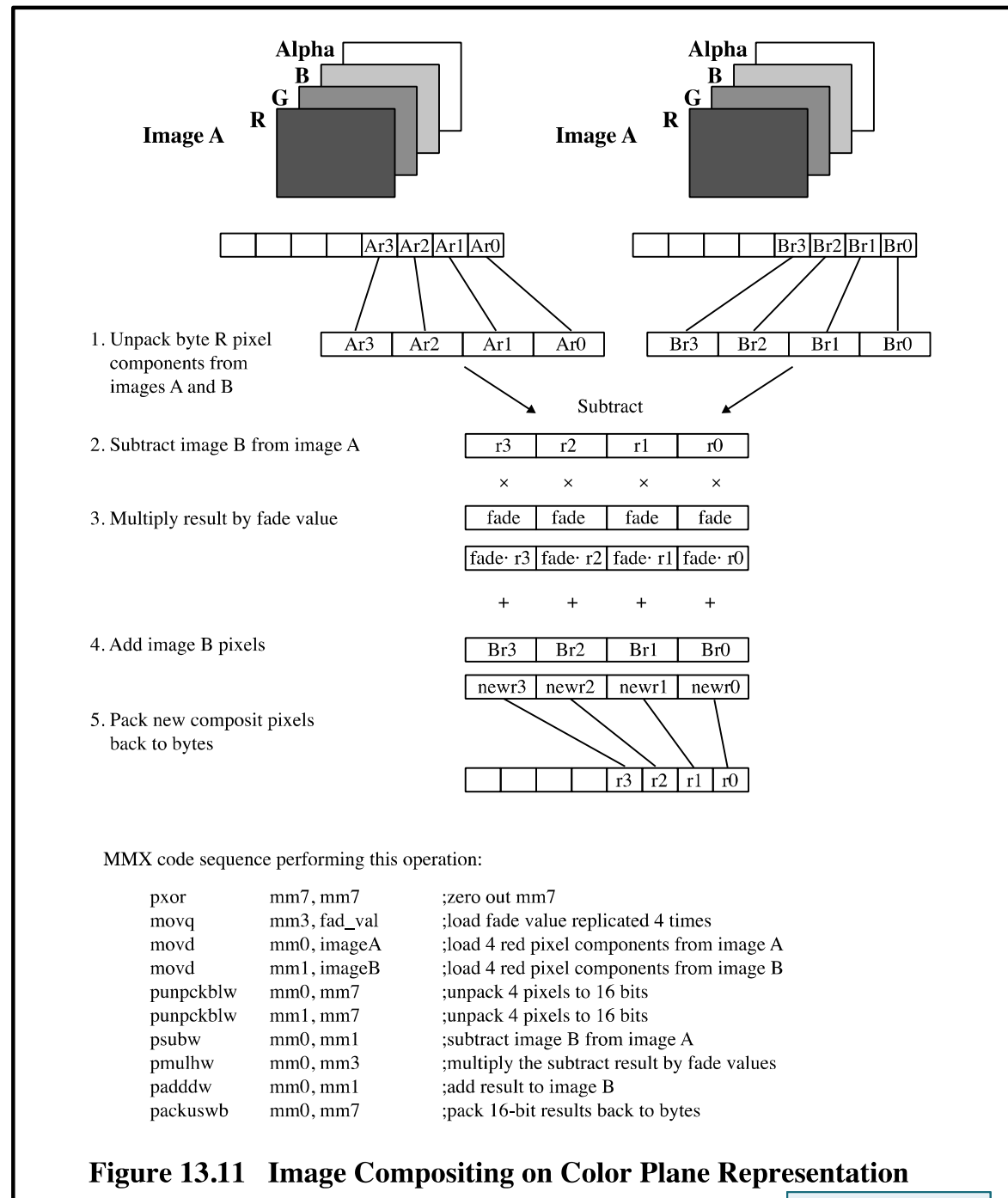


Figure 13.11 Image Compositing on Color Plane Representation

ARM Operation Types

Load and store
instructions

Branch
instructions

Data-processing
instructions

Multiply
instructions

Parallel addition
and subtraction
instructions

Extend
instructions

Status register
access
instructions

Table 13.11

ARM Conditions for Conditional Instruction Execution

Code	Symbol	Condition Tested	Comment
0000	EQ	$Z = 1$	Equal
0001	NE	$Z = 0$	Not equal
0010	CS/HS	$C = 1$	Carry set/unsigned higher or same
0011	CC/LO	$C = 0$	Carry clear/unsigned lower
00100	MI	$N = 1$	Minus/negative
00101	PL	$N = 0$	Plus/positive or zero
00110	VS	$V = 1$	Overflow
00111	VC	$V = 0$	No overflow
1000	HI	$C = 1 \text{ AND } Z = 0$	Unsigned higher
1001	LS	$C = 0 \text{ OR } Z = 1$	Unsigned lower or same
1010	GE	$N = V$ [[$(N = 1 \text{ AND } V = 1)$ OR $(N = 0 \text{ AND } V = 0)$]	Signed greater than or equal
1011	LT	$N \neq V$ [[$(N = 1 \text{ AND } V = 0)$ OR $(N = 0 \text{ AND } V = 1)$]	Signed less than
1100	GT	$(Z = 0) \text{ AND } (N = V)$	Signed greater than
1101	LE	$(Z = 1) \text{ OR } (N \neq V)$	Signed less than or equal
1110	AL	–	Always (unconditional)
1111	–	–	This instruction can only be executed unconditionally

Figure 14.2

x86 Addressing Mode Calculation

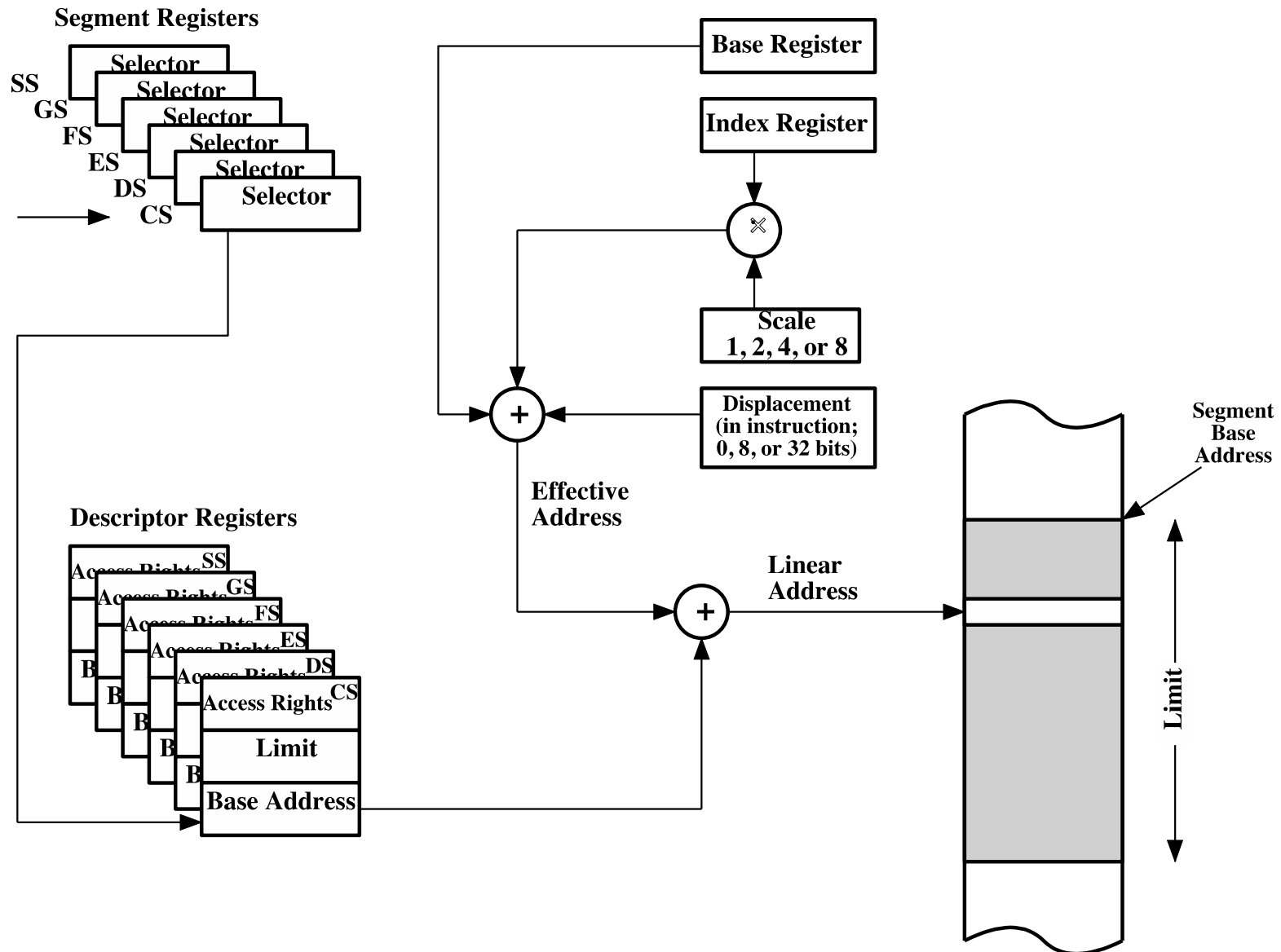


Table 14.2

x86 Addressing Modes

Mode	Algorithm
Immediate	Operand = A
Register Operand	LA = R
Displacement	LA = (SR) + A
Base	LA = (SR) + (B)
Base with Displacement	LA = (SR) + (B) + A
Scaled Index with Displacement	LA = (SR) + (I) × S + A
Base with Index and Displacement	LA = (SR) + (B) + (I) + A
Base with Scaled Index and Displacement	LA = (SR) + (I) × S + (B) + A
Relative	LA = (PC) + A

LA = linear address

(X) = contents of X

SR = segment register

PC = program counter

A = contents of an address field in the instruction

R = register

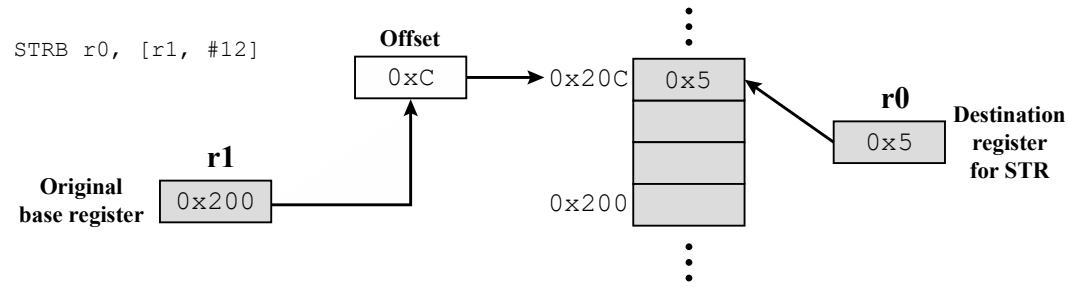
B = base register

I = index register

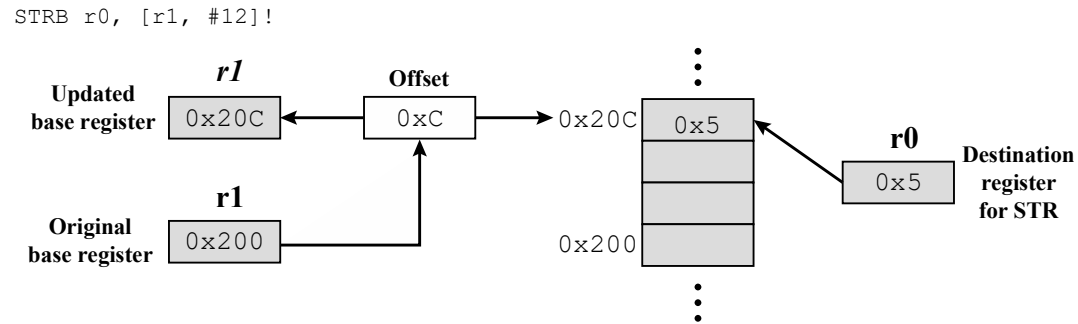
S = scaling factor

Figure 14.3

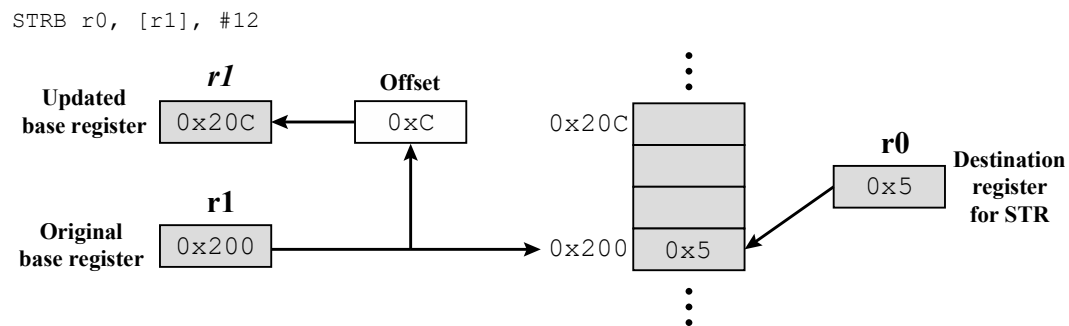
ARM Indexing Methods



(a) Offset



(b) Preindex



(c) Postindex

ARM Data Processing Instruction Addressing and Branch Instructions

- Data processing instructions
 - Use either register addressing or a mixture of register and immediate addressing
 - For register addressing the value in one of the register operands may be scaled using one of the five shift operators
- Branch instructions
 - The only form of addressing for branch instructions is immediate
 - Instruction contains 24 bit value
 - Shifted 2 bits left so that the address is on a word boundary
 - Effective range $\pm 32\text{MB}$ from from the program counter

Figure 14.4

ARM Load/Store Multiple Addressing

```
LDMxx r10, {r0, r1, r4}  
STMxx r10, {r0, r1, r4}
```

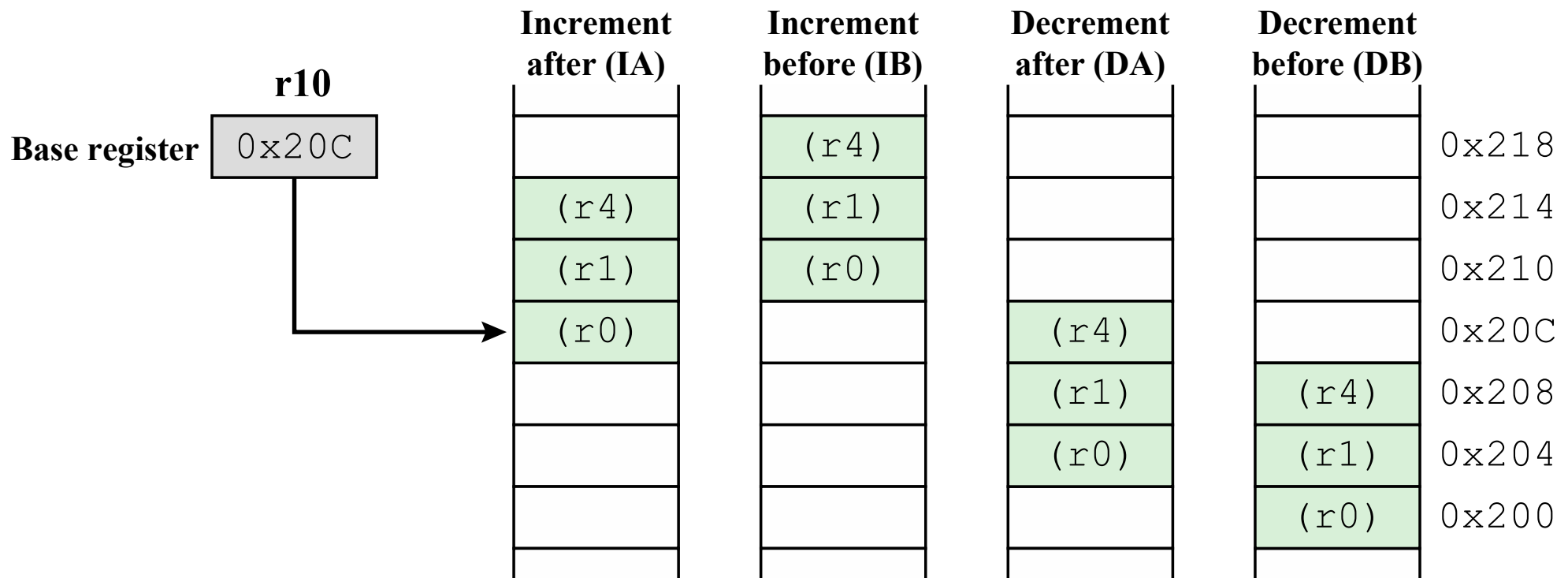


Figure 14.5

PDP-8 Instruction Formats

Memory Reference Instructions

Opcode		D/I	Z/C	Displacement						
0	2	3	4	5						11

Input/Output Instructions

1	1	0	Device					Opcode		
0	2	3					8	9		11

Register Reference Instructions

Group 1 Microinstructions

1	1	1	0	CLA	CLL	CMA	CML	RAR	RAL	BSW	IAC
0	1	2	3	4	5	6	7	8	9	10	11

Group 2 Microinstructions

1	1	1	1	CLA	SMA	SZA	SNL	RSS	OSR	HLT	0
0	1	2	3	4	5	6	7	8	9	10	11

Group 3 Microinstructions

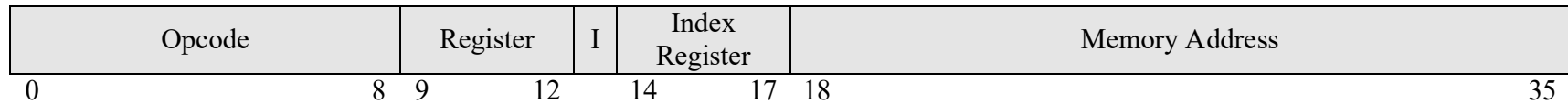
1	1	1	1	CLA	MQA	0	SQL	0	0	0	1
0	1	2	3	4	5	6	7	8	9	10	11

D/I = Direct/Indirect address
 Z/C = Page 0 or Current page
 CLA = Clear Accumulator
 CLL = Clear Link
 CMA = CoMplement Accumulator
 CML = CoMplement Link
 RAR = Rotate Accumulator Right
 RAL = Rotate Accumulator Left
 BSW = Byte SWap

IAC = Increment ACcumulator
 SMA = Skip on Minus Accumulator
 SZA = Skip on Zero Accumulator
 SNL = Skip on Nonzero Link
 RSS = Reverse Skip Sense
 OSR = Or with Switch Register
 HLT = HaLT
 MQA = Multiplier Quotient into Accumulator
 SQL = Multiplier Quotient Load

Figure 14.6

PDP-10 Instruction Format



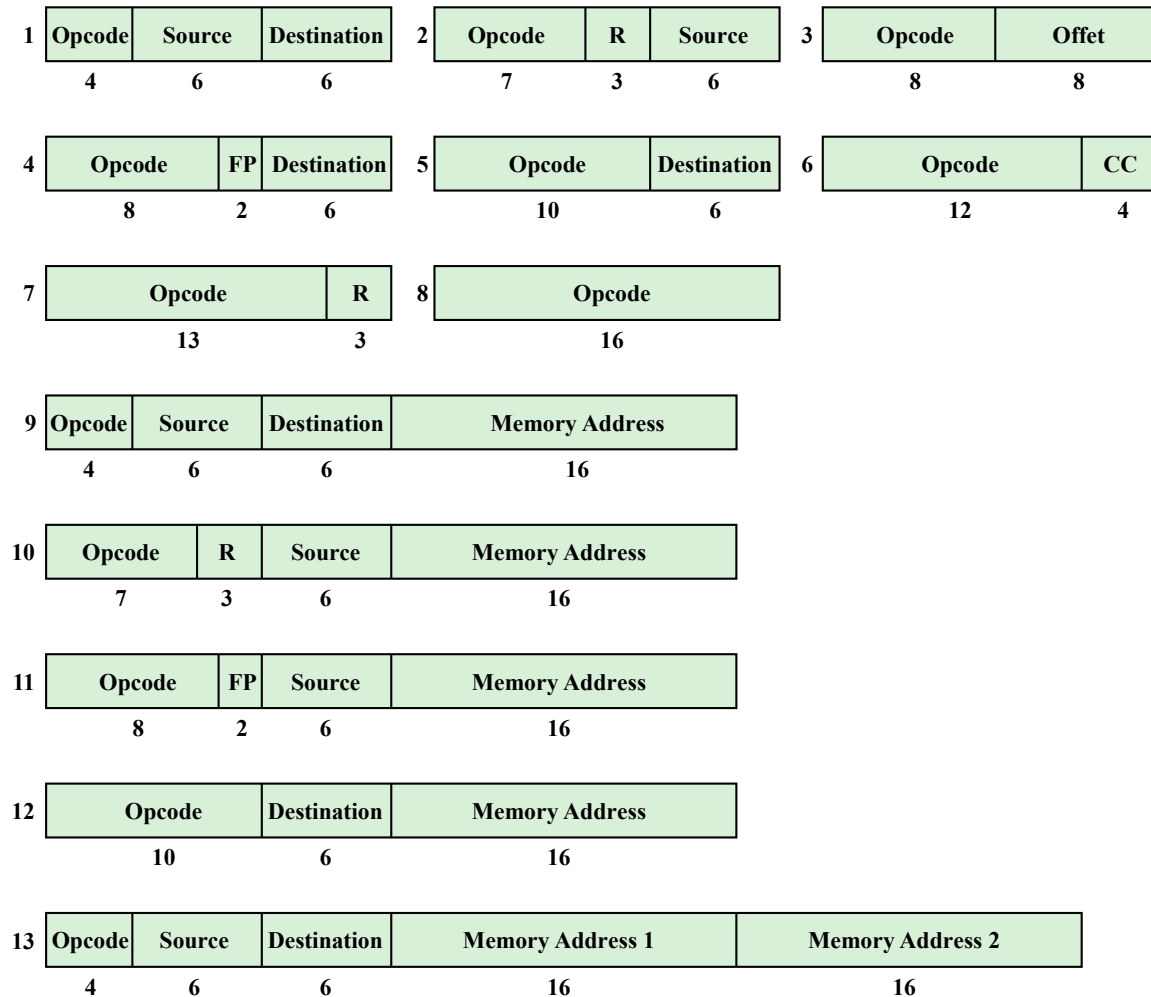
I = indirect bit

Variable-Length Instructions

- Variations can be provided efficiently and compactly
- Increases the complexity of the processor
- Does not remove the desirability of making all of the instruction lengths integrally related to word length
 - Because the processor does not know the length of the next instruction to be fetched a typical strategy is to fetch a number of bytes or words equal to at least the longest possible instruction
 - Sometimes multiple instructions are fetched

Figure 14.7

Instruction Formats for the PDP-11



Numbers below fields indicate bit length

Source and Destination each contain a 3-bit addressing mode field and a 3-bit register number

FP indicates one of four floating-point registers

R indicates one of the general-purpose registers

CC is the condition code field

Figure 14.8

Example of VAX Instructions

Hexadecimal Format	Explanation	Assembler Notation and Description
8 bits 05	Opcode for RSB	RSB Return from subroutine
D4 59	Opcode for CLRL Register R9	CLRL R9 Clear register R9
B0 C4 64 01 A B 1 9	Opcode for MOVW Word displacement mode, Register R4 356 in hexadecimal Byte displacement mode, Register R11 25 in hexadecimal	MOVW 356(R4), 25(R11) Move a word from address that is 356 plus contents of R4 to address that is 25 plus contents of R11
C1 05 50 42 D F	Opcode for ADDL3 Short literal 5 Register mode R0 Index prefix R2 Indirect word relative (displacement from PC) Amount of displacement from PC relative to location A	ADDL3 #5, R0, @A[R2] Add 5 to a 32-bit integer in R0 and store the result in location whose address is sum of A and 4 times the contents of R2

Figure 14.9

x86 Instruction Format

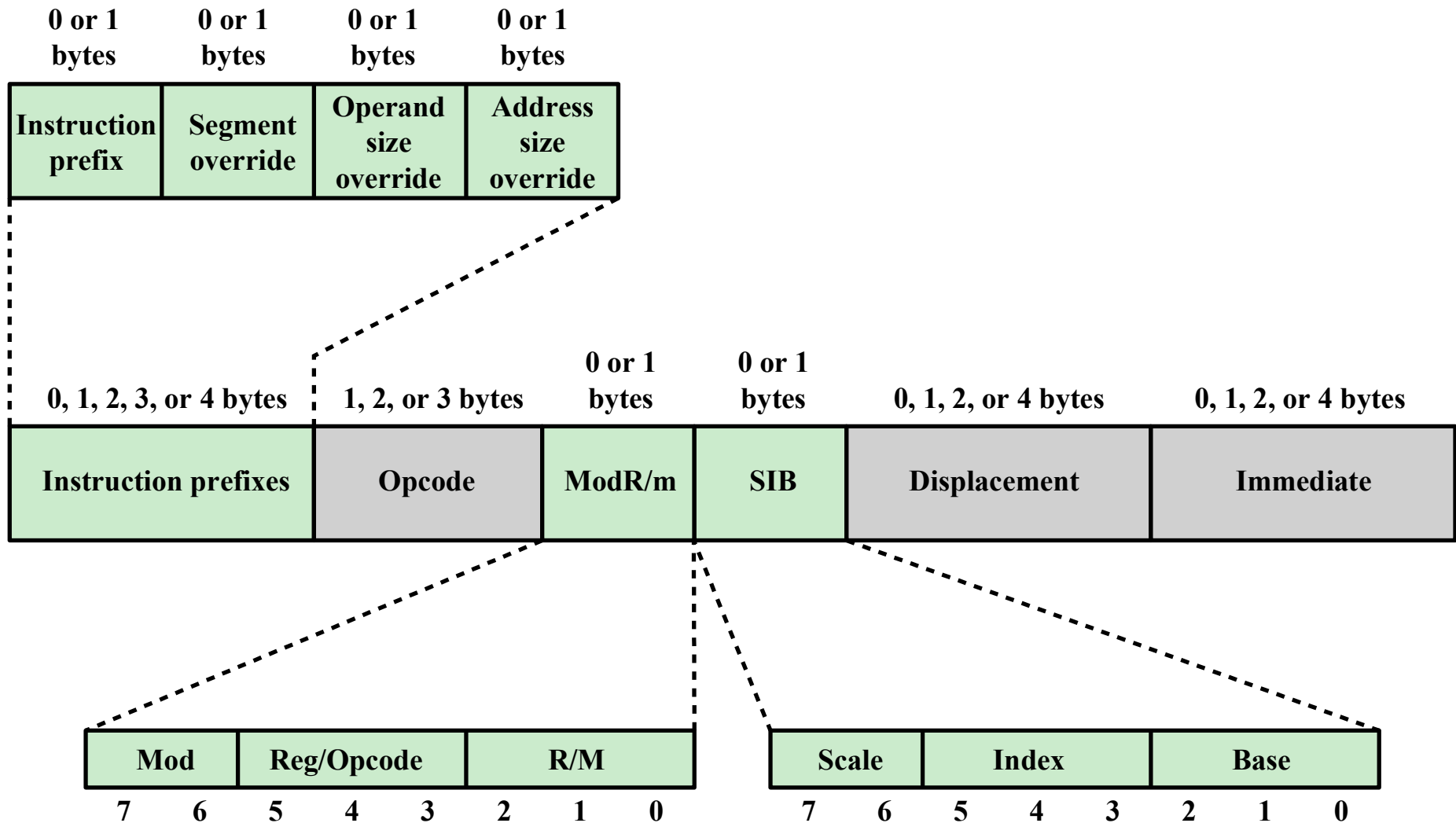


Figure 14.10

ARM Instruction Formats

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
data processing immediate shift	cond		0 0 0			opcode			S	Rn			Rd			shift amount				shift		0	Rm									
data processing register shift	cond		0 0 0			opcode			S	Rn			Rd			Rs			0	shift		1	Rm									
data processing immediate	cond		0 0 1			opcode			S	Rn			Rd			rotate			immediate													
load/store immediate offset	cond		0 1 0			P	U	B	W	L	Rn			Rd			immediate															
load/store register offset	cond		0 1 1			P	U	B	W	L	Rn			Rd			shift amount				shift		0	Rm								
load/store multiple	cond		1 0 0			P	U	S	W	L	Rn			register list																		
branch/branch with link	cond		1 0 1			L	24-bit offset																									

S = For data processing instructions, signifies that the instruction updates the condition codes

S = For load/store multiple instructions, signifies whether instruction execution is restricted to supervisor mode

P, U, W = bits that distinguish among different types of addressing_mode

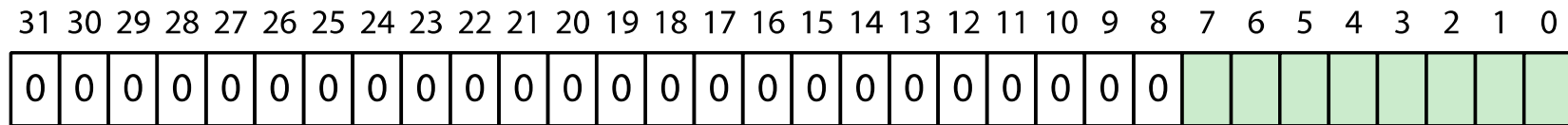
B = Distinguishes between an unsigned byte (B==1) and a word (B==0) access

L = For load/store instructions, distinguishes between a Load (L==1) and a Store (L==0)

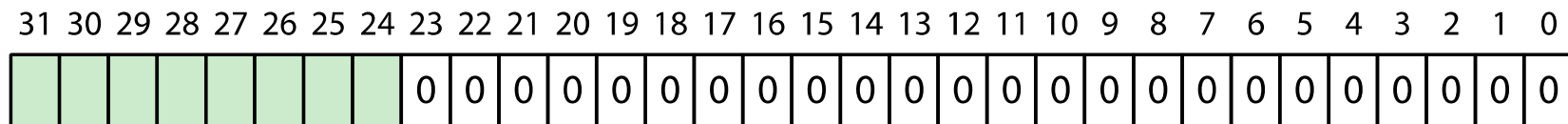
L = For branch instructions, determines whether a return address is stored in the link register

Figure 14.11

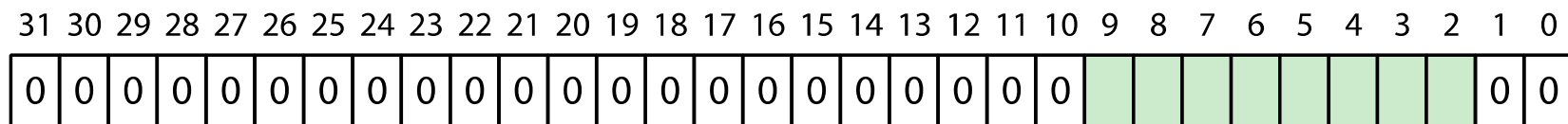
Examples of Use of ARM Immediate Constants



ror #—0 range 0 through 0x000000FF—step 0x00000001



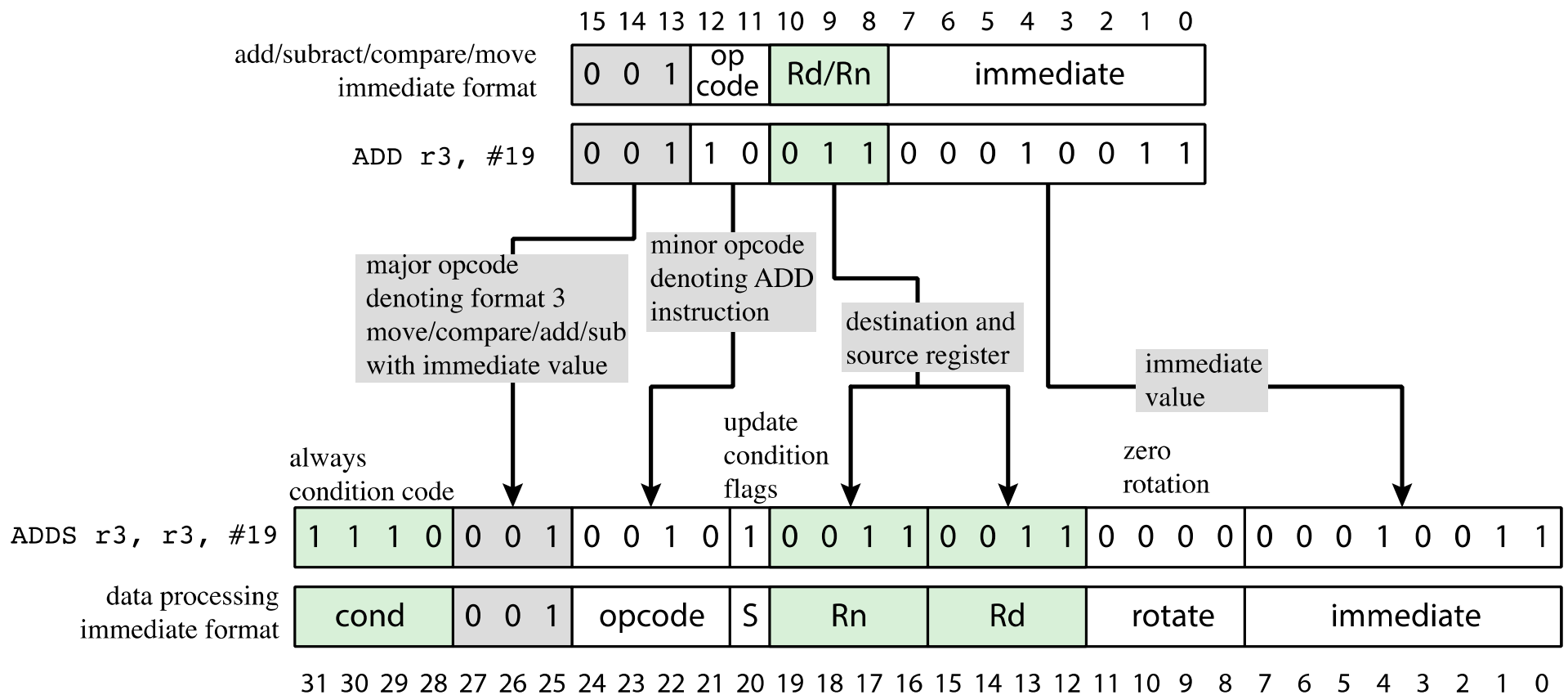
ror #8—range 0 through 0xFF000000—step 0x01000000



ror #30—range 0 through 0x000003FC—step 0x00000004

Figure 14.12

Expanding a Thumb ADD Instruction into its ARM Equivalent

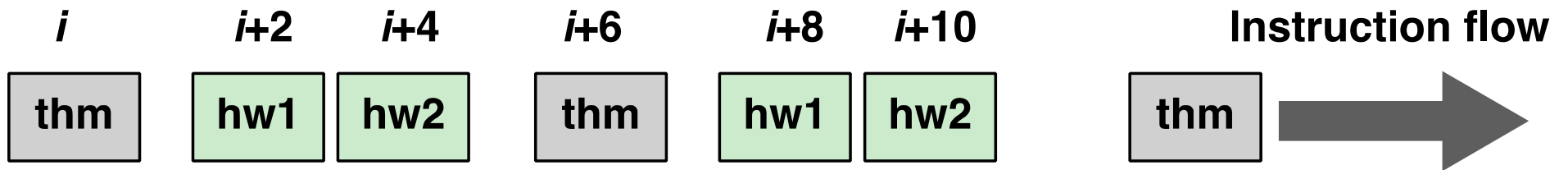


Thumb-2 Instruction Set

- The only instruction set available on the Cortex-M microcontroller products
- Is a major enhancement to the Thumb instruction set architecture (ISA)
 - Introduces 32-bit instructions that can be intermixed freely with the older 16-bit Thumb instructions
 - Most 32-bit Thumb instructions are unconditional, whereas almost all ARM instructions can be conditional
 - Introduces a new If-Then (IT) instruction that delivers much of the functionality of the condition field in ARM instructions
- Delivers overall code density comparable with Thumb, together with the performance levels associated with the ARM ISA
- Before Thumb-2 developers had to choose between Thumb for size and ARM for performance

Figure 14.13

Thumb-2 Encoding



Halfword 1 [15:13]	Halfword1 [12:11]	Length	Functionality
Not 111	xx	16 bits (1 halfword)	16-bit Thumb instruction
111	00	16 bits (1 halfword)	16-bit Thumb unconditional branch instruction
111	Not 00	32 bits (2 halfwords)	32-bit Thumb-2 instruction

Summary

Chapter 13

- Machine instruction characteristics
 - Elements of a machine instruction
 - Instruction representation
 - Instruction types
 - Number of addresses
 - Instruction set design
- Types of operands
 - Numbers
 - Characters
 - Logical data

Instruction Sets: Characteristics and Functions

- Intel x86 and ARM data types
- Types of operations
 - Data transfer
 - Arithmetic
 - Logical
 - Conversion
 - Input/output
 - System control
 - Transfer of control
- Intel x86 and ARM operation types

Summary

Chapter 14

- Addressing modes
 - Immediate addressing
 - Direct addressing
 - Indirect addressing
 - Register addressing
 - Register indirect addressing
 - Displacement addressing
 - Stack addressing

Instruction Sets: Addressing Modes and Formats

- x86 addressing modes
- ARM addressing modes
- Instruction formats
 - Instruction length
 - Allocation of bits
 - Variable-length instructions
- X86 instruction formats
- ARM instruction formats

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructions in teaching their courses and assessing student learning. dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.